

Introducere în Godot

(Convenție – scrisul portocaliu are hyperlink-uri embedded către documentație, dacă dați click/ CTRL+click o să vă redirecționeze către documentația aferentă)

Prof. Coordonator: Ș.I.dr.ing.Stelian-Nicolae NICOLĂ

Mentor: Pătrașcu Hary

Ce sisteme vom realiza până la finalul modului 2D?

- Movement
- Enemies(cu navigare)
- NPCs(cu dialog și interacțiuni)
- Shop(economie)
- Tilemaps(pentru crearea hărților)
- Farming
- Elemente de UI(meniuri, etc)
- Inventory
- Mining

Ce este **Godot**



Un game engine open-source, cross-platform cu capabilități 2D și 3D dezvoltat în C++.

Godot permite folosirea a mai multor limbaje de programare(principal GDScript și C#) și publicarea jocurilor pe toate sistemele de operare moderne: Windows, Linux, MacOS, Android, iOS, dar și pe console.

GScript



GScript este un limbaj de programare high-level, orientat pe obiecte cu sintaxa similară limbajului Python.

Exemple de cod în GDScript

```
Debug      player_controller.gd      Online Docs  Search Help  < >

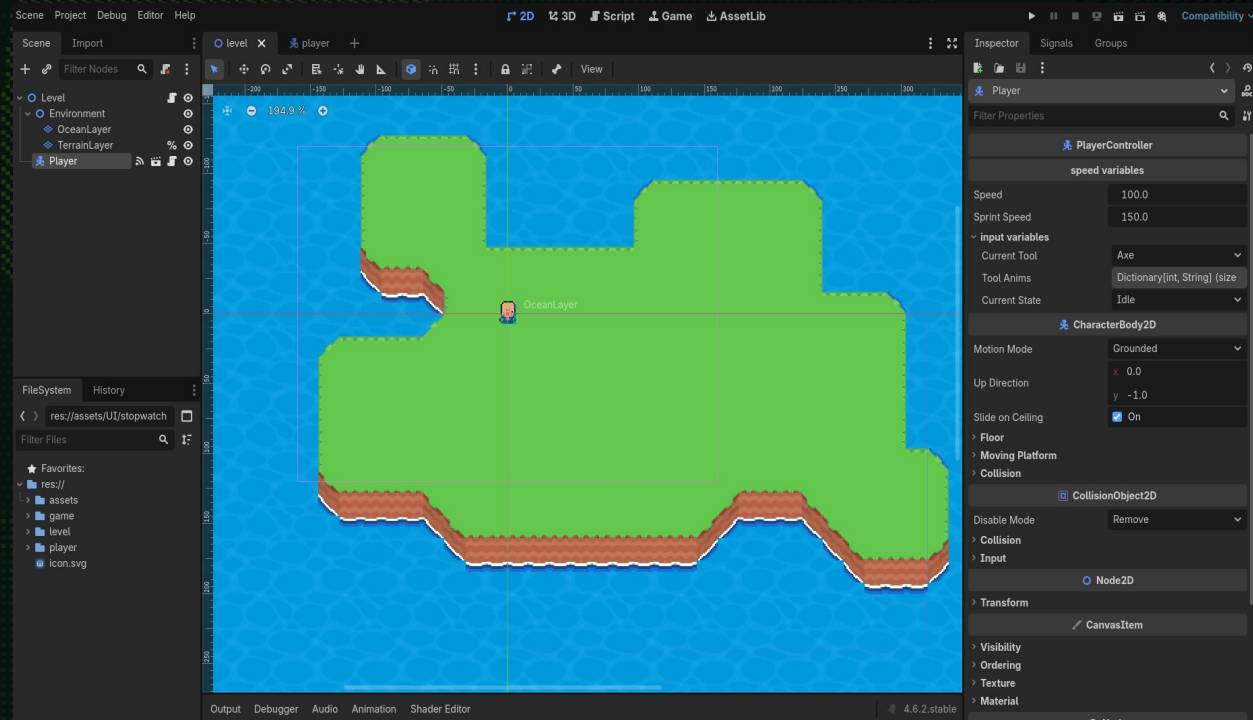
1  class_name PlayerController extends CharacterBody2D
2
3
4  @onready var tools_animated_sprite: AnimatedSprite2D = $ToolsAnimatedSprite
5  @onready var character_animated_sprite: AnimatedSprite2D = $CharacterAnimatedSprite
6
7  @export_category("speed variables")
8  @export var _speed: float = 100.0
9  @export var _sprint_speed: float = 150.0
10 var _current_speed: float = _speed
11
12 @export_group("input variables")
13 var _input_dir: Vector2 = Vector2.ZERO
14 var _can_move: bool = true
15
16 ▾ var _facing_right: bool = false:
17   ▾ >| set(value):
18     >| >| _facing_right = value
19     >| >| character_animated_sprite.flip_h = not _facing_right
20     >| >| tools_animated_sprite.flip_h = not _facing_right
21
22 > enum Tool {
23   29 }
24
25 @export var _current_tool: Tool
26 @export var _tool_anims: Dictionary[Tool, String]
27
28 ▾ enum State {
29   34 >| IDLE,
30   35 >| WALKING,
31   36 >| RUNNING,
32   37 >| USING_TOOL
33   38 }
```

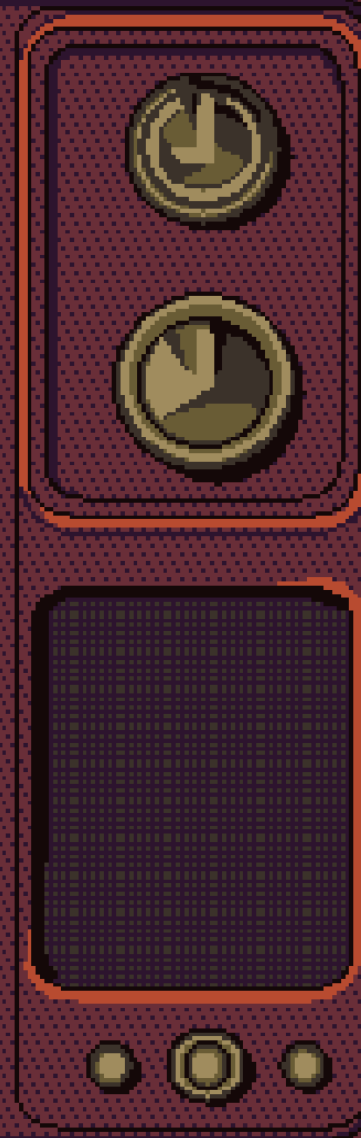
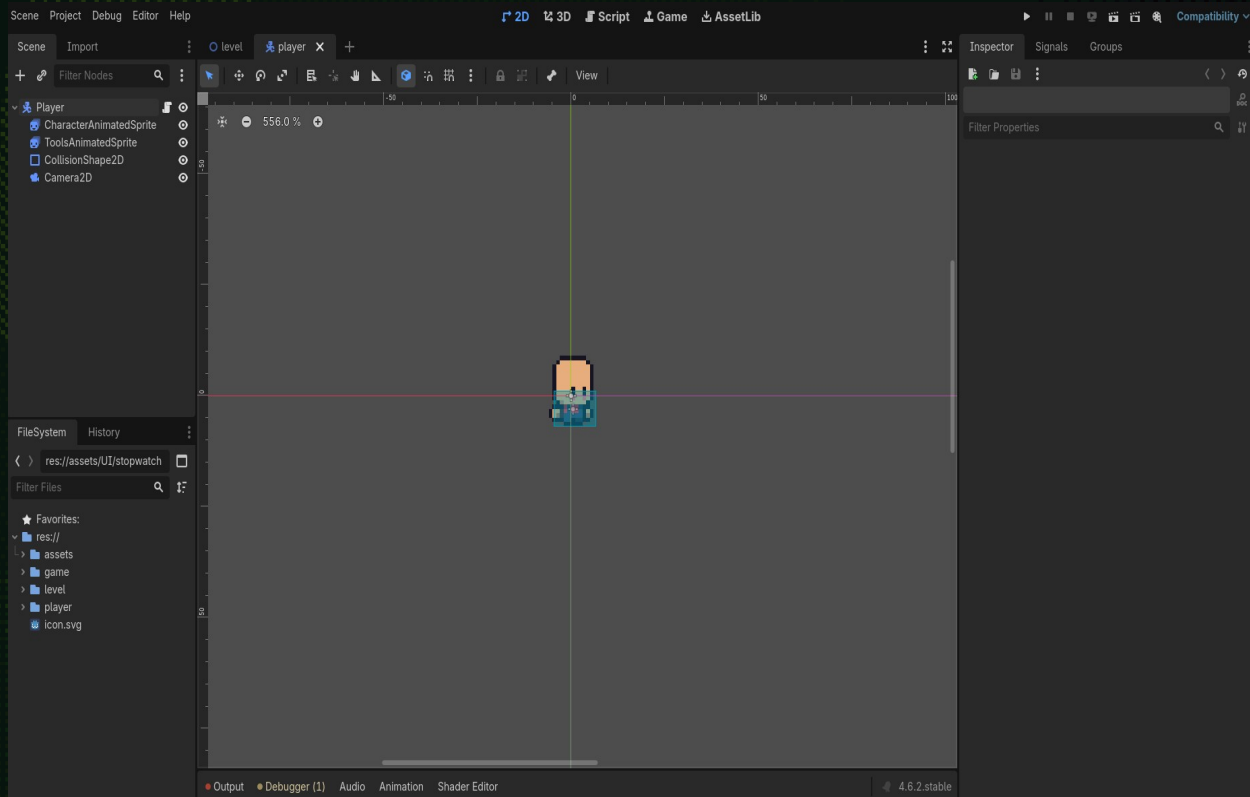

Scene

Jocurile făcute în Godot sunt structurate folosind scene reutilizabile. O scenă poate să fie un caracter, un obiect cu care poți interacționa, un meniu, un nivel întreg, etc.

Scenele pot fi imbricate, astfel putem pune scenele unui caracter, inamici și obiecte într-o scenă nivel.

Exemple de scene





Noduri

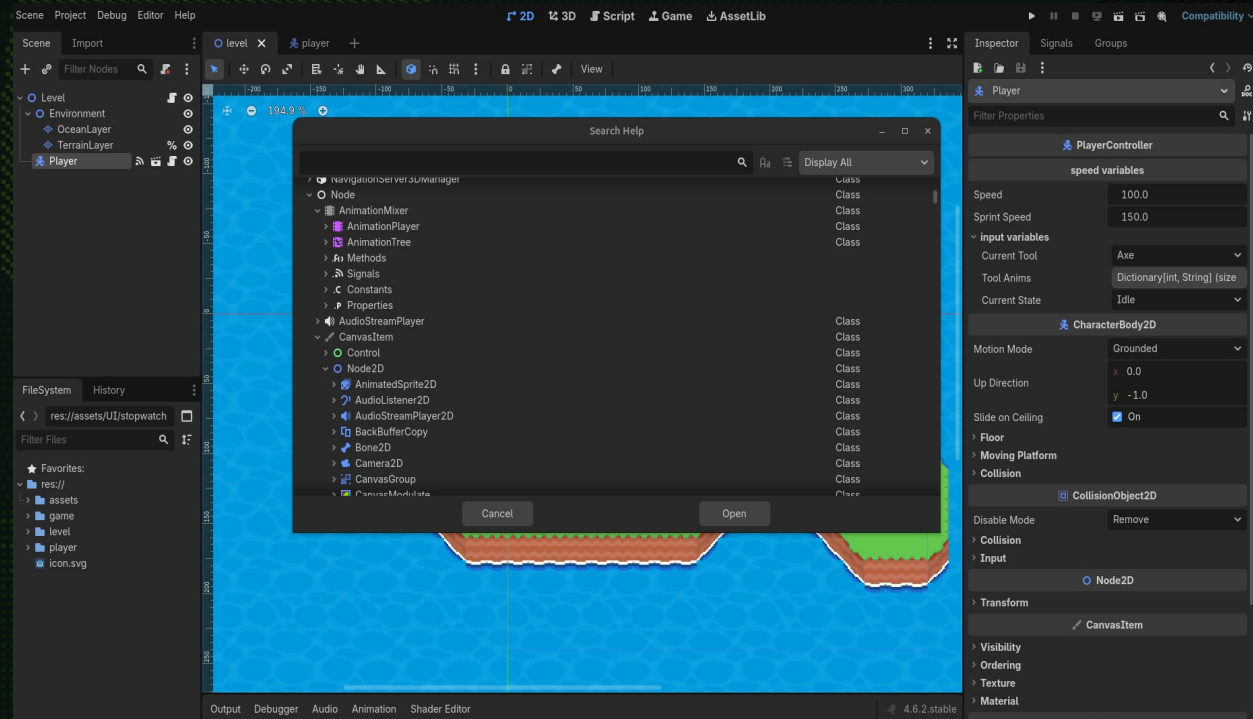
O scenă este compusă din noduri.

Nodurile în Godot sunt cele mai mici obiecte de sine stătătoare pe care le aranjezi astfel încât să compui o scenă.

Prin atașarea unui script unui nod, putem să îi extindem funcționalitatea cu propriul nostru cod.

Putem accesa meniul help folosind tasta F1 sau RCLICK > open documentation pe un nod pentru a afla mai multe informații.

Exemple de noduri



SceneProject Debug Editor Help

2D3DScriptGameAssetLib

SceneImport

Filter Nodes

LevelEnvironmentOceanLayerTerrainLayerPlayer

FileSystemHistory

res://assets/UI/stopwatch

Filter Files

Assetsgamelevelplayericon.svg

FileSearch Debug

CharacterBody2D

Filter Scripts

level.gdplayer_controller.gdArrayCharacterBody2D

InspectorSignals Groups

Player

Filter Properties

PlayerController

speed variables

Speed100.0

Sprint Speed150.0

input variables

Current ToolAxe

Tool AnimDictionary[int, String] (size)

Current StateIdle

CharacterBody2D

Motion ModeGrounded

Up Direction

x0.0

y-1.0

Slide on Ceiling

Floor

Moving Platform

Collision

CollisionObject2D

Disable ModeRemove

Collision

Input

Node2D

Transform

CanvasItem

Visibility

Ordering

Texture

Material

Class: **CharacterBody2D**

Inherits: PhysicsBody2D < CollisionObject2D < Node2D < CanvasItem < Node < Object

Inherited by: PlayerController

A 2D physics body specialized for characters moved by script.

Description

CharacterBody2D is a specialized class for physics bodies that are meant to be user-controlled. They are not affected by physics at all, but they affect other physics bodies in their path. They are mainly used to provide high-level API to move objects with wall and slope detection (`move_and_slide()` method) in addition to the general collision detection provided by `PhysicsBody2D.move_and_collide()`. This makes it useful for highly configurable physics bodies that must move in specific ways and collide with the world, as is often the case with user-controlled characters.

For game objects that don't require complex movement or collision detection, such as moving platforms, `AnimateableBody2D` is simpler to configure.

Online Tutorials

Physics introduction

Troubleshooting physics issues

Kinematic character (2D)

Using CharacterBody2D

2D_Kinematic_Character_Demo

2D_Platformer_Demo

Properties

bool floor_block_on_wall [default: true]

bool floor_constant_speed [default: false]

float floor_max_angle [default: 0.7853982]

Output Debugger Audio Animation Shader Editor

4.6.2.stable



Semnale

Semnalele sunt folosite pentru a facilita comunicarea între noduri. De exemplu când un caracter atinge un obiect putem să transmitem un semnal pentru a crește un scor.

Există de asemenea și semnale built-in pentru noduri pe care le putem vizualiza în engine(de exemplu un buton când este apăsat emite un semnal).

În plus, semnalele sunt folosite pentru a implementa design patterns și programare orientată pe obiecte, despre care vom vorbi la următoarele ședințe.

Exemple de semnale

Class: ➔ Signal

A built-in type representing a signal of an Object.

Description

Signal is a built-in Variant type that represents a signal of an Object instance. Like all Variant types, it can be stored in variables and passed to functions. Signals allow all connected Callable (and by extension their respective objects) to listen and react to events, without directly referencing one another. This keeps the code flexible and easier to manage. You can check whether an Object has a given signal name using `Object.has_signal()`.

In GDScript, signals can be declared with the `signal` keyword. In C#, you may use the `[Signal]` attribute on a delegate.

```
signal attacked
```

```
# Additional arguments may be declared.  
# These arguments must be passed when the signal is emitted.  
signal item_dropped(item_name, amount)
```

Connecting signals is one of the most common operations in Godot and the API gives many options to do so, which are described further down. The code block below shows the recommended approach.

```
func _ready():  
    var button = Button.new()  
    # 'button_down' here is a Signal Variant type. We therefore  
    call the Signal.connect() method, not Object.connect().
```

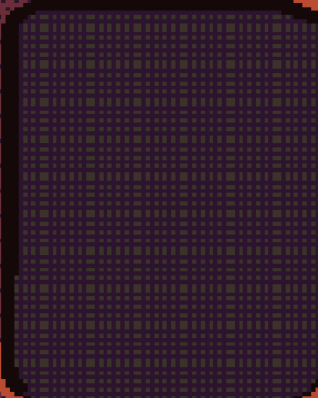
```
PlayerController  
└─ tool_used(tool: PlayerController.Tool, pos: Vector2)  
CollisionObject2D  
└─ input_event(viewport: Node, event: InputEvent, shape_idx: int)  
└─ mouse_entered()  
└─ mouse_exited()  
└─ mouse_shape_entered(shape_idx: int)  
└─ mouse_shape_exited(shape_idx: int)  
CanvasItem  
└─ draw()  
└─ hidden()  
└─ item_rect_changed()  
└─ visibility_changed()  
Node  
└─ child_entered_tree(node: Node)  
└─ child_exiting_tree(node: Node)  
└─ child_order_changed()  
└─ editor_description_changed(node: Node)  
└─ editor_state_changed()  
└─ ready()  
└─ renamed()  
└─ replacing_by(node: Node)  
└─ tree_entered()  
└─ tree_exited()  
└─ tree_exiting()  
Object  
└─ property_list_changed()  
└─ script_changed()
```

Animation Shader Editor

4.6.2.stable

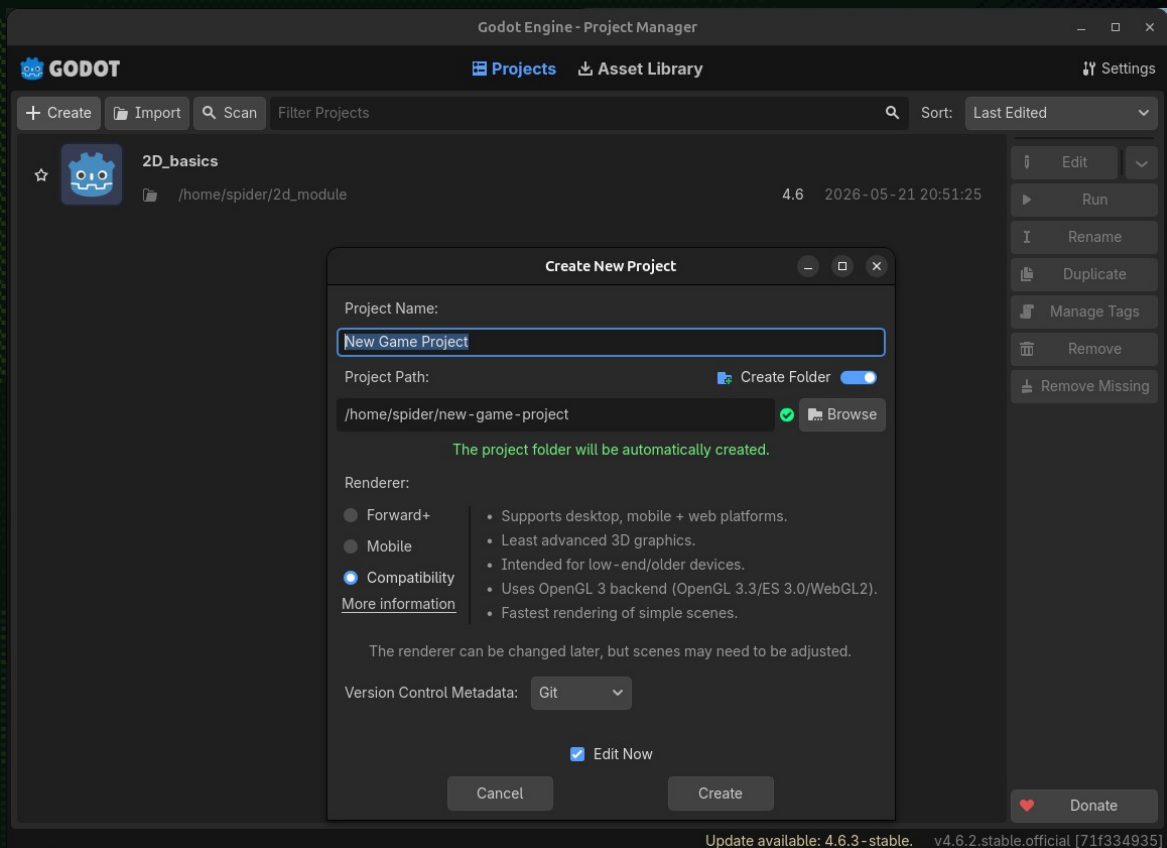
Connect...

```
1  extends Node2D
2
3  @onready var player: PlayerController = $"../Player"
4  signal new_environment_created()
5
6
7  # Called when the node enters the scene tree for the first time.
8  func _ready() -> void:
9      # listen for emit() from the player scene and call _on_tool_used() when received
10     player.tool_used.connect(_on_tool_used)
11
12     # broadcast signal to listeners
13     new_environment_created.emit()
14
15  func _on_tool_used() -> void:
16     pass
17
```



Interfața și navigarea

Când deschideți Godot, veți vedea project manager-ul unde puteți crea proiecte, instala proiecte demo din asset library și schimba setările de aspect ale engine-ului.

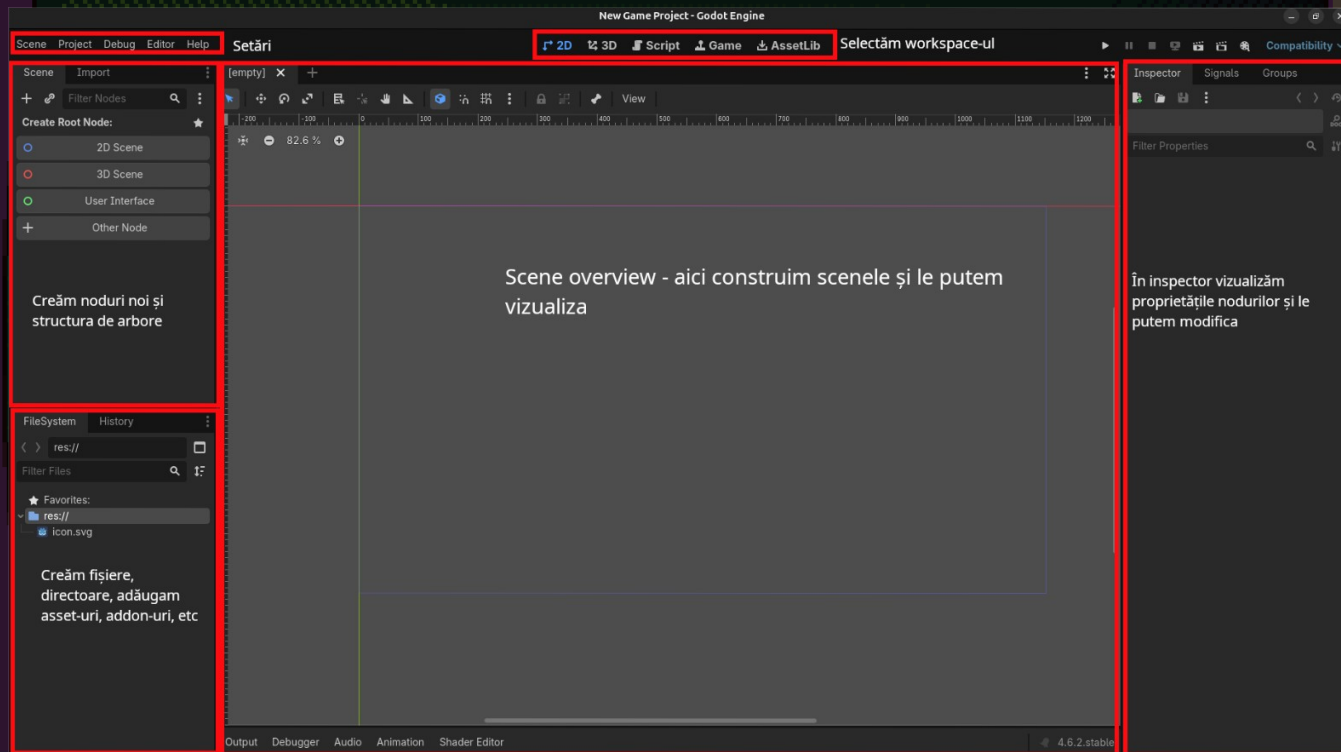


Moduri de rendering

Godot dispune de trei moduri de rendering:

- Forward+ – folosit pentru jocuri cu grafici high-end/ realiste, optimizat pentru scene complexe;
- Mobile – folosit pentru dispozitive mobile și desktop, afișează scene simple mai rapid.
- Compatibility – folosit pentru toate platformele, inclusiv web deoarece este gândit pentru dispozitive low-end.

Vom folosi compatibility pentru a putea exporta jocul și pe web.(în general, dacă este posibil, este bine să faci export pe web deoarece oamenii nu trebuie să downloadeze jocul și o să ai parte de o audiență mai mare)



VCS: lucrul cu **git** și **GitHub**

VCS(version control system) este software care ține un istoric al schimbărilor dintr-un proiect. Cel mai folosit VCS este git.

Avantajul folosirii unui VCS este că simplifică lucrul în echipă și creează backup-uri ale proiectului. Platforma cea mai folosită pentru a hosta un repository(un proiect software) este GitHub.

Requirements: git instalat pe sistem, cont de GitHub



GitHub Desktop

GitHub Desktop este un software care simplifică lucrul cu git pentru utilizatorii Windows/MacOS și există un **fork de Linux**.

Tutorial GitHub

Top repositories

New

- 0xspiderr/gbjam13_gamejam
- 0xspiderr/awawa-game-steam
- 0xspiderr/multiplayer-fps
- 0xspiderr/itec-2026
- 0xspiderr/godot-visual-prog-fps
- 0xspiderr/enet-chess
- 0xspiderr/esp32-cam-prj

Show more

Pe butonul
'New' creăm un
repository nou

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *

 Oxspiderr ▾

Repository name *

Great repository names are short and memorable. How about [urban-goggles?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

Create repository

Adăugăm setări
Pentru repository

test_tutorial Private

Watch 0 Fork 0 Star 0



Start coding with Codespaces

Add a README file and start coding in a secure, configurable, and dedicated development environment.

Create a codespace



Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

HTTPS SSH

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# test_tutorial" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/oxspiderr/test_tutorial.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/oxspiderr/test_tutorial.git
git branch -M main
git push -u origin main
```

ProTip! Use the URL for this page when adding GitHub as a remote.

Setup
repository(cu
GitHub
desktop tot
procesul
acesta este
automatizat)

Comenzi git(optional pentru cine vrea)

În folder-ul proiectului nostru trebuie să creem un repository git local.
Pentru asta trebuie să deschidem command line-ul(pe Windows git bash).



FileSystem

History

< > res://

Filter Files

★ Favorites:

▼ res://

🖼 icon.svg

+ Create New

📁 Expand Folder

▼ Expand Hierarchy

▶ Collapse Hierarchy

📄 Copy Path

Ctrl+Shift+C

Copy Absolute Path

Ctrl+Alt+C

★ Add to Favorites

➤ Open in Terminal

Ctrl+Alt+T

📂 Open in File Manager

Ctrl+Alt+R

Output

Debugger

Aut



spider@spider: ~/github-tutorial/



```
spider@spider:~/github-tutorial/$ git init
```

hint: Using 'master' as the name for the initial branch. This default branch name

hint: is subject to change. To configure the initial branch name to use in all

hint: of your new repositories, which will suppress this warning, call:

hint:

hint: git config --global init.defaultBranch <name>

hint:

hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and

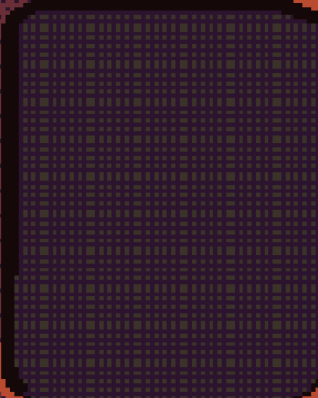
hint: 'development'. The just-created branch can be renamed via this command:

hint:

hint: git branch -m <name>

Initialized empty Git repository in /home/spider/github-tutorial/.git/

```
spider@spider:~/github-tutorial/$ git remote add origin https://github.com/0xspiderr/test_tutorial
```



Câteva comenzi importante de git

`git init` – crează un repository local

`git remote add origin <url>` - conectează repository-ul nostru local cu locul unde este hostat(GitHub în cazul nostru).

`origin` – numele default al repository-ul remote



```
spider@spider:~/github-tutorial/$ git status
```

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
.editorconfig
```

```
.gitattributes
```

```
.gitignore
```

```
icon.svg
```

```
icon.svg.import
```

```
project.godot
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

```
spider@spider:~/github-tutorial/$ git add .
```


`git status` - afișează branch-ul curent pe care ne aflăm și alte detalii.

Un branch pe git este un workspace separat pe care putem lucra la alte feature-uri și este foarte util pentru lucrul în echipă. În general workflow-ul pe care l-am folosit este să avem un branch separat fiecare membru din echipă și să îl sincronizăm cu branch-ul principal(main).

`git add <nume_fișier>` - adaugă fișiere modificate pe care urmează să le commit-uim

`git add .` - pentru a adăuga toate fișierele

```
spider@spider:~/github-tutorial/$ git commit -m "first commit"
```

```
[main (root-commit) 1a57092] first commit
```

```
6 files changed, 78 insertions(+)
```

```
create mode 100644 .editorconfig
```

```
create mode 100644 .gitattributes
```

```
create mode 100644 .gitignore
```

```
create mode 100644 icon.svg
```

```
create mode 100644 icon.svg.import
```

```
create mode 100644 project.godot
```

```
spider@spider:~/github-tutorial/$ git push -u origin main
```

```
Enumerating objects: 8, done.
```

```
Counting objects: 100% (8/8), done.
```

```
Delta compression using up to 16 threads
```

```
Compressing objects: 100% (6/6), done.
```

```
Writing objects: 100% (8/8), 1.86 KiB | 1.86 MiB/s, done.
```

```
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0
```

```
To https://github.com/0xspiderr/test_tutorial
```

```
* [new branch]      main -> main
```

```
branch 'main' set up to track 'origin/main'.
```

```
spider@spider:~/github-tutorial/$
```

`git commit -m <mesaj de commit>` - crează un "snapshot" al proiectului și adaugă un mesaj. În general ar trebui să punem mesaje bazat pe feat-urile adăugate sau acțiunile făcute(delete/update, etc).

`git push -u origin main` - publică schimbările(commit-urile) pe remote(GitHub).

Argumentele `-u origin main` sunt necesare doar la primul commit, la următorul commit putem scrie direct:

`git push`

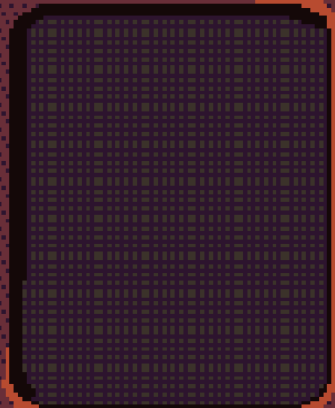
Alte comenzi utile

`git pull` - preia toate schimbările de pe un repository remote. Util când lucrezi cu mai mulți oameni sau ai mai multe device-uri de pe care lucrezi și vrei să le sincronizezi. De asemenea folosit pentru sincronizare de pe alte branch-uri.

`git help <nume_comandă>` - afișează text ajutător despre comanda dată ca argument

`git clone <url>` - downloadează local un repository remote(puteți să dați clone la repository-ul cu prezentări)

`git clone https://github.com/0xspiderr/gdc-upt-presentations/tree/main`



Workflow git

1. Pașii de inițializare(îi facem o singură dată):

```
git init
```

```
git remote add origin <url>
```

```
git add .
```

```
Git commit -m <mesaj>
```

```
git push -u origin main
```

2. Pașii în development

```
git pull
```

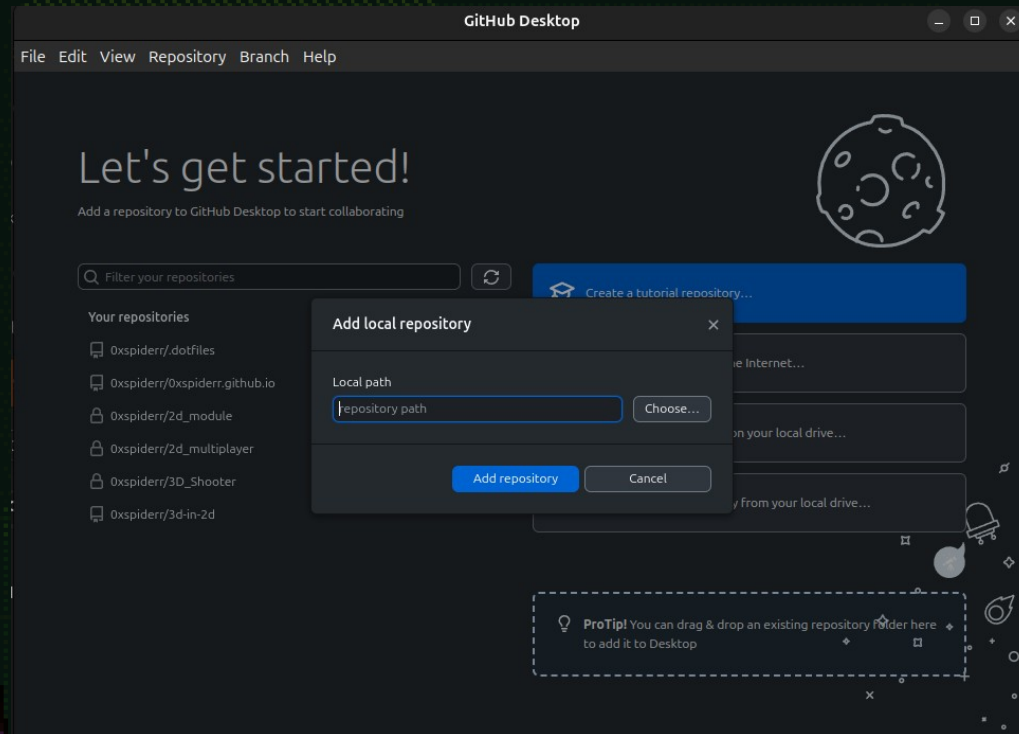
```
git add .
```

```
git commit -m <mesaj>
```

```
git push
```

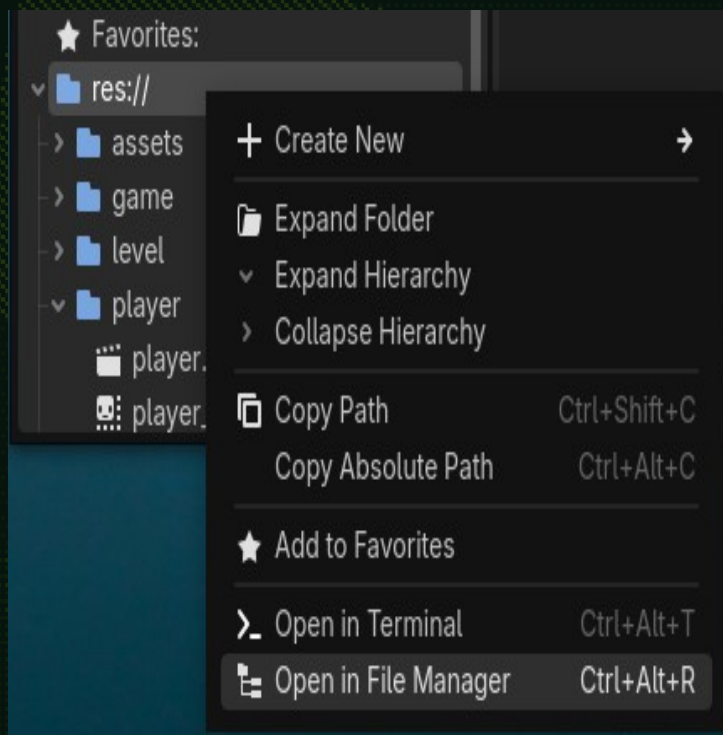
Dacă lucrați solo, o să vă descurcați 99% din timp doar cu aceste 4 comenzi. Este esențial să dăm git pull de fiecare dată când scriem o comandă dacă mai avem alte device-uri de pe care lucrăm/ lucrăm cu alți oameni pe același branch, dacă nu o să avem merge conflicts.

Github Desktop tutorial

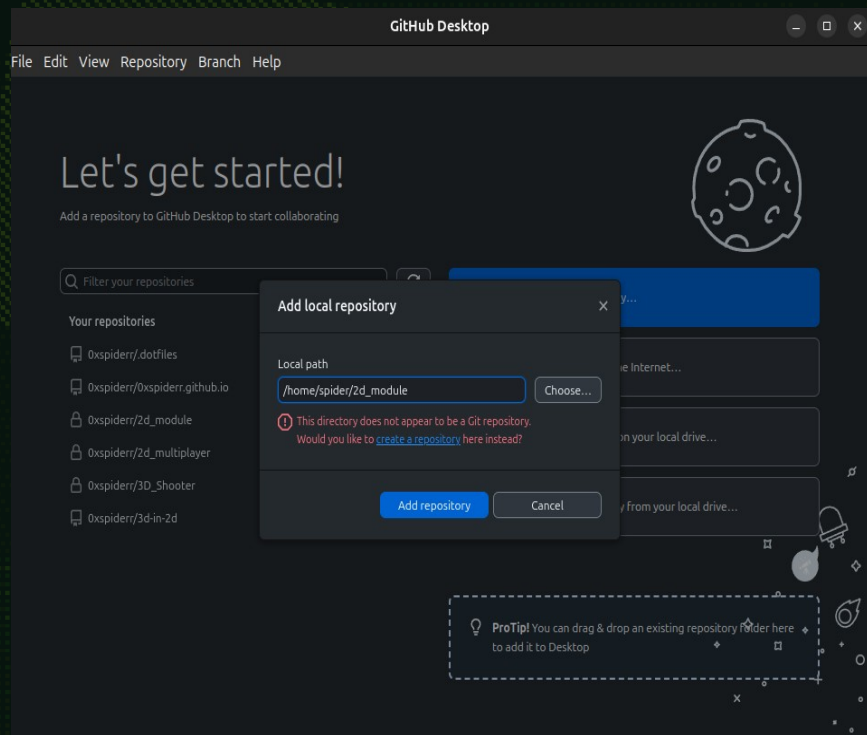


File →
Add local
repository

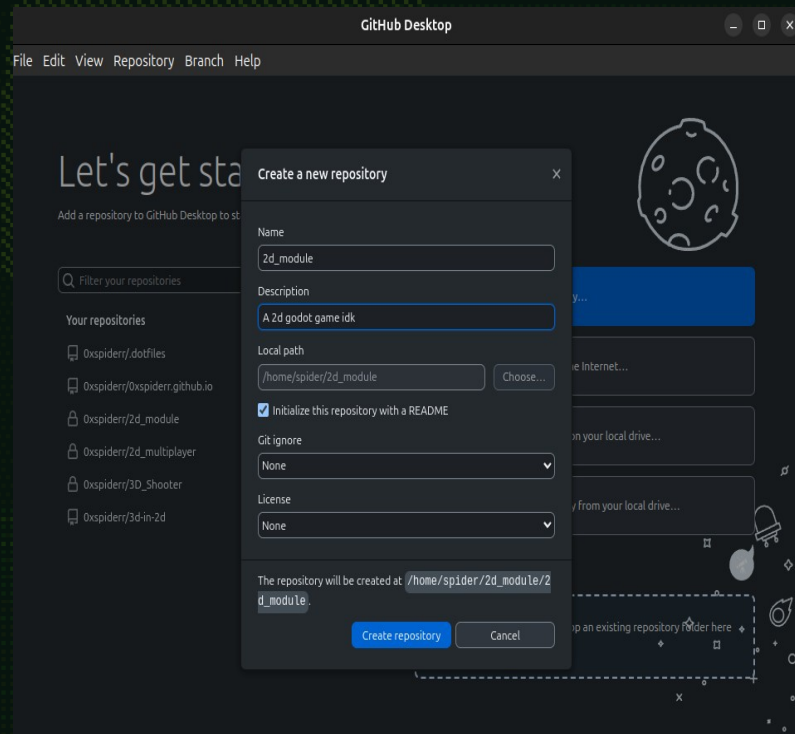




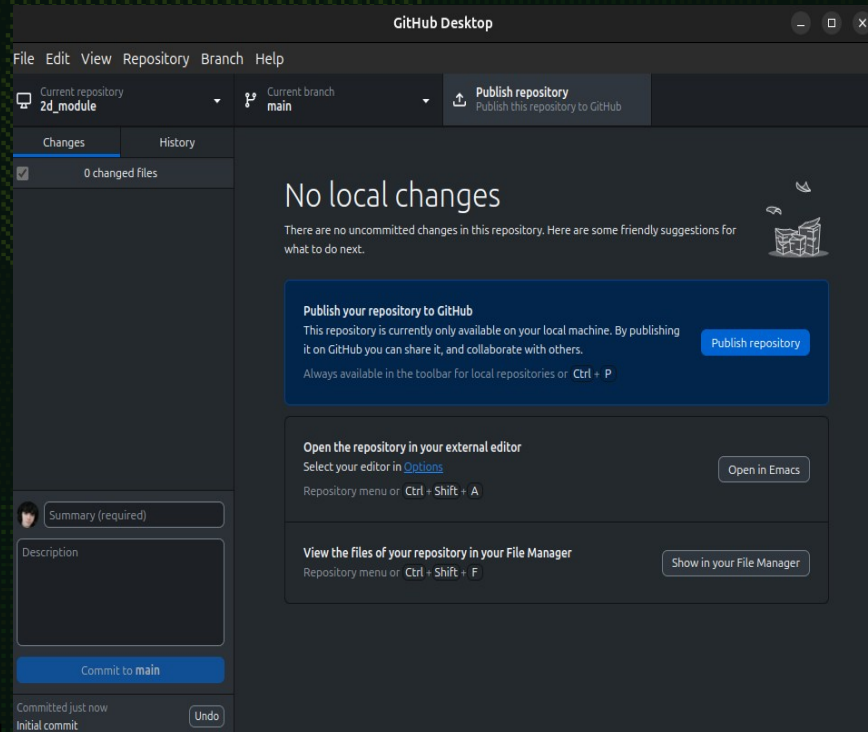
**Deschideți proiectul
din file manager și copiați
path-ul proiectului.**



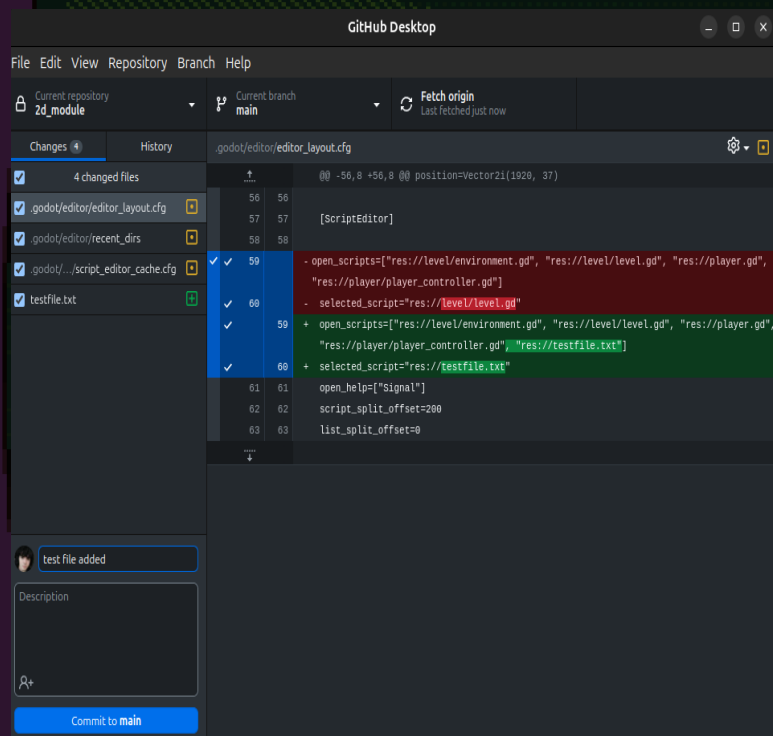
Dați click pe linkul
'create a repository'



Adăugați setările de
care aveți nevoie și
dați click pe
'create repository'



Publicați repository-ul pe GitHub



În partea stângă se află fișierele modificate din proiect și jos puteți să dați commit apoi push.

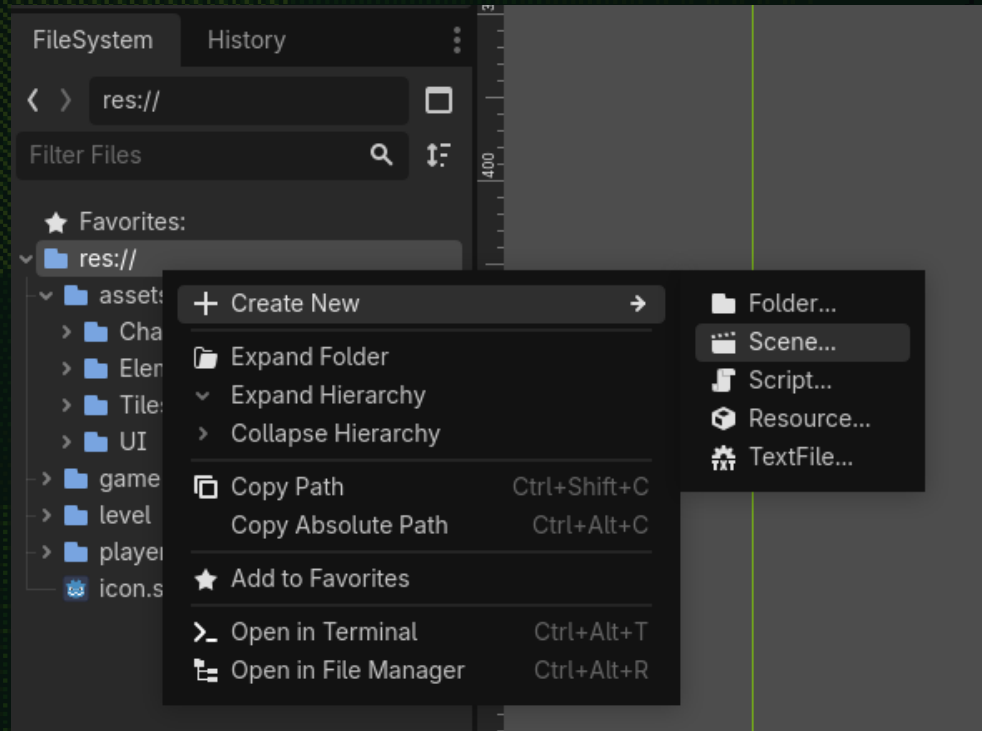
Butonul de fetch/ pull îl folosiți pentru a updata repository-ul de pe remote când lucrați de pe mai multe device-uri sau în echipă. Când lucrați în echipă este recomandat să creați branch-uri separate pentru fiecare coechipier pentru a evita merge conflicts.

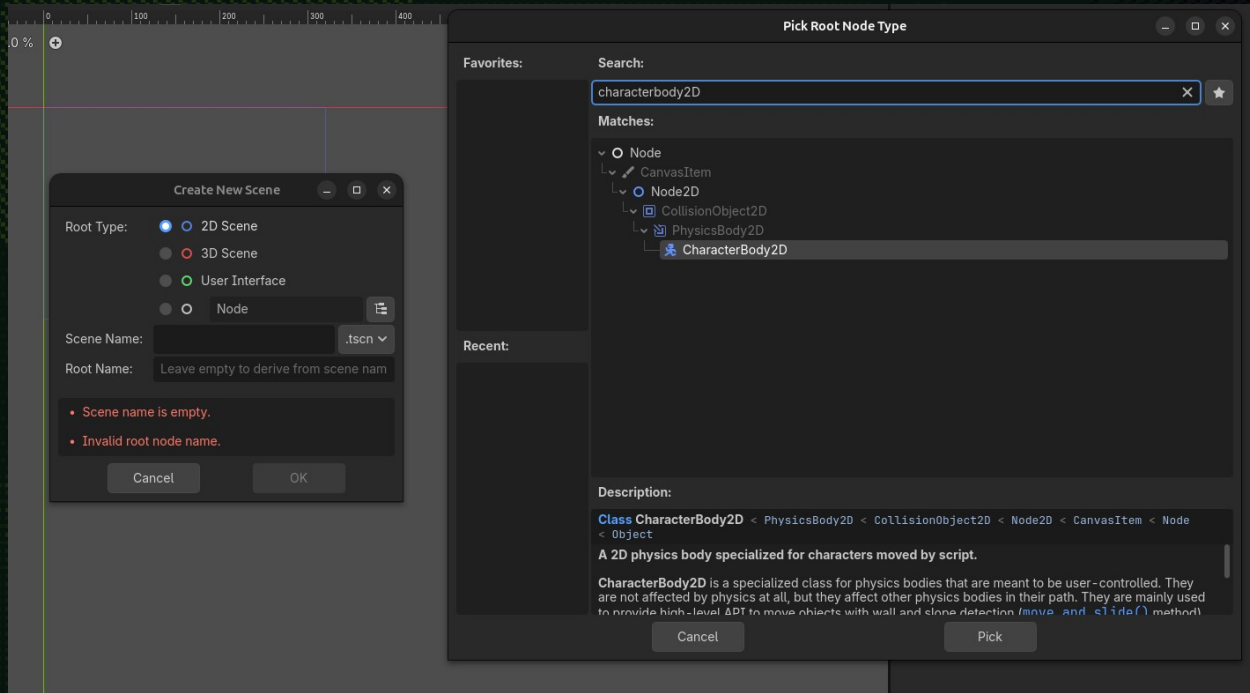
Avem un proiect Godot și un repository de git pe GitHub, să începem development-ul



Crearea nodurilor și scenelor în Godot

În primul rând, avem nevoie de o scenă care să conțină noduri.





Creăm un caracter principal pe care jucătorul îl va controla, în Godot avem un nod special pentru asta: `CharacterBody2D`

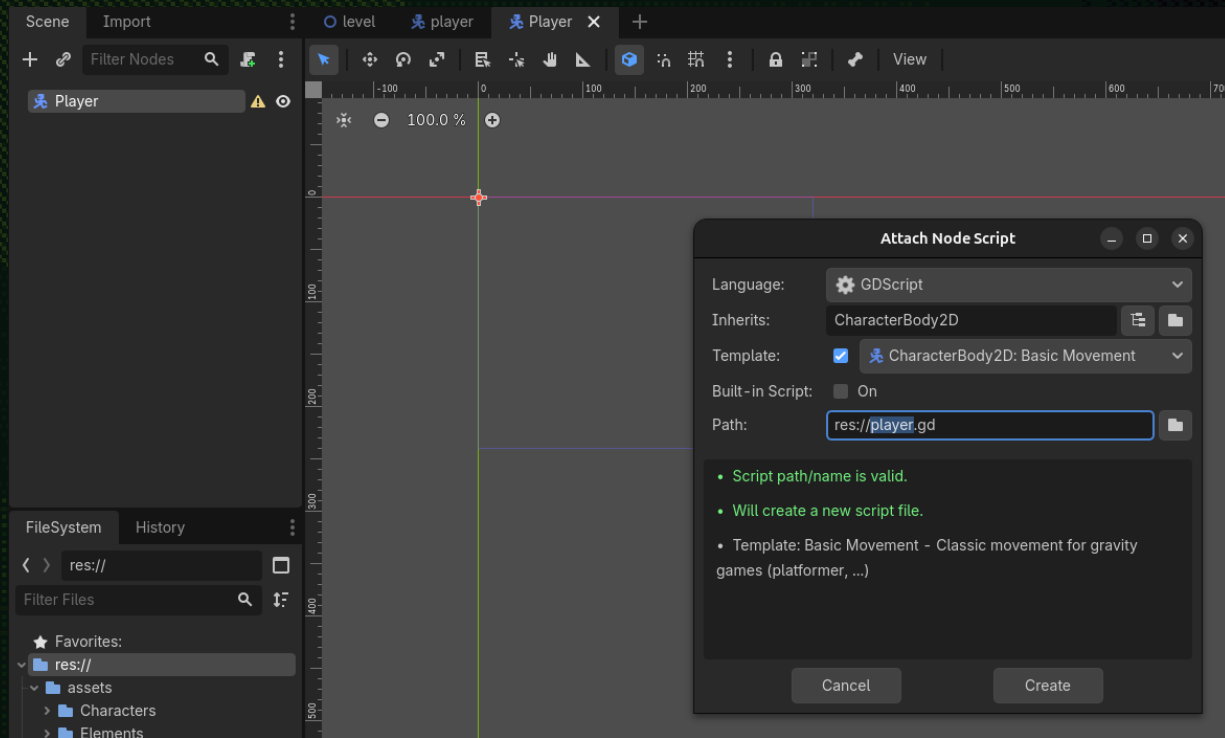
Scena pe care o creem are nevoie de un nod părinte și un nume. Nodul părinte este nodul care va conține toate celelalte noduri și le poate controla din script-uri.

După creare, trebuie să salvăm scena într-un folder. Scenele create au extensia `.tscn`.

Scripting

Pentru a controla player-ul, trebuie să atașăm un script nodului pentru a putea să scriem cod.

Cum atașăm un script unui nod?





Există mai multe modalități.

- Rclick pe nod → attach script
- Click pe nod → click pe iconița cu un script și un + verde

După ce ați dat un nume script-ului și l-ați creat, workspace-ul de scripting se va deschide.

```

1  extends CharacterBody2D
2
3
4  const SPEED = 300.0
5  const JUMP_VELOCITY = -400.0
6
7
8  func _physics_process(delta: float) -> void:
9      # Add the gravity.
10     if not is_on_floor():
11         velocity += get_gravity() * delta
12
13     # Handle jump.
14     if Input.is_action_just_pressed("ui_accept") and is_on_floor():
15         velocity.y = JUMP_VELOCITY
16
17     # Get the input direction and handle the movement/deceleration.
18     # As good practice, you should replace UI actions with custom gameplay actions.
19     var direction := Input.get_axis("ui_left", "ui_right")
20     if direction:
21         velocity.x = direction * SPEED
22     else:
23         velocity.x = move_toward(velocity.x, 0, SPEED)
24
25     move_and_slide()
26

```

```

1  # Add your code here
2
3  # Add your code here
4
5  # Add your code here
6
7  # Add your code here
8
9  # Add your code here
10
11 # Add your code here
12
13 # Add your code here
14
15 # Add your code here
16
17 # Add your code here
18
19 # Add your code here
20
21 # Add your code here
22
23 # Add your code here
24
25 # Add your code here
26

```

```

1  # Add your code here
2
3  # Add your code here
4
5  # Add your code here
6
7  # Add your code here
8
9  # Add your code here
10
11 # Add your code here
12
13 # Add your code here
14
15 # Add your code here
16
17 # Add your code here
18
19 # Add your code here
20
21 # Add your code here
22
23 # Add your code here
24
25 # Add your code here
26

```

Sistemul de fizică

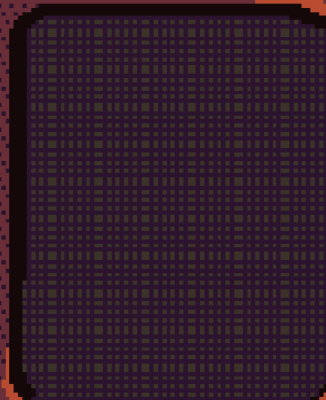
În Godot pentru jocuri în 2D există 4 tipuri de corpuri fizice:

- Area2D -> folosit pentru a detecta când anumite noduri se intersectează și emit un semnal de intrare sau ieșire.
- StaticBody2D -> folosit pentru obiecte statice din lume în care ai avea nevoie doar de collision detection de exemplu.
- RigidBody2D -> noduri care implementează fizică și pe care poți aplica forțe, impulsuri, gravitație etc.
- CharacterBody2D -> Un nod care implementează collision detection, dar nu implementează fizica(codul de fizică trebuie scris de programator).

De asemenea vom avea nevoie de un nod care să detecteze când corpurile se ciocnesc(collision detection). Pentru asta avem collision shapes. În cazul nostru nodul **CollisionShape2D**.

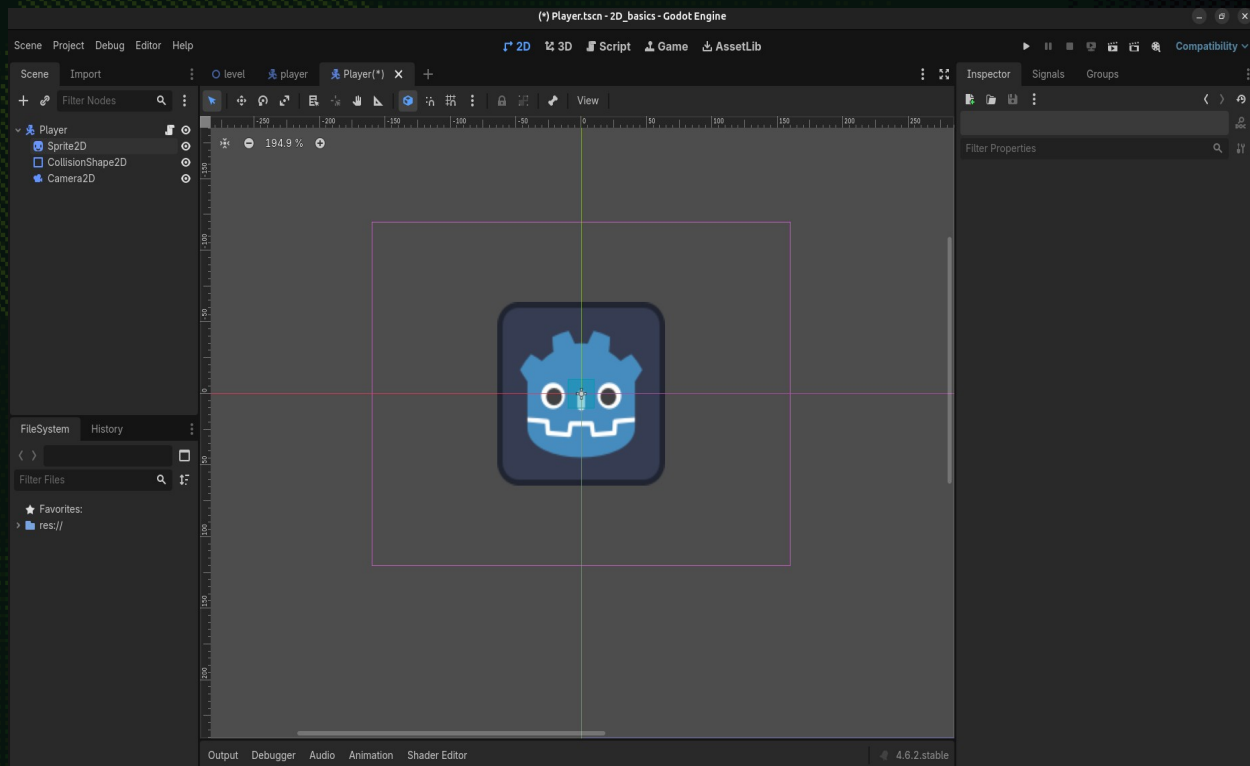
Pentru a vizualiza caracterul avem nodul **Sprite2D/AnimatedSprite2D**.

În final putem adăuga o cameră pentru a urmări caracterul când acesta se mișcă folosind nodul **Camera2D**.



Exercițiu, încercați să adăugați în ordine nodurile enumerate pe slide-ul anterior părintelui 'Player':

- Sprite2D → redenumiți nodul acesta 'PlayerVisualComponent' și adăugați resursa icon.svg ca textură din inspector
- CollisionShape2D → Adăugați un collision shape din inspector(panelul din dreapta)
- Camera2D → creșteți/ scădeți proprietatea zoom până când arată cum vă place.



Să mișcăm caracterul!

De ce credeți că am avea nevoie?

Răspuns:

- Input de la tastatură
- Poziția jucătorului
- Viteza cu care vrem să se miște jucătorul

Sistemul de input

Există două modalități de a obține input în Godot. Dar cea mai folosită este prin InputMap și clasa globală Input.

InputMap permite definirea unor acțiuni mapate la taste/mouse/controller sau orice alt periferic de intrare.

Pentru a defini acțiuni accesăm meniul Project > Project Settings > Input Map

Project Settings (project.godot)

General

Input Map

Localization

Globals

Plugins

Import Defaults

Filter by Name

Filter by Event

Clear All

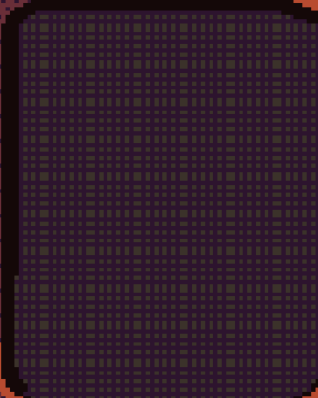
Add New Action

+ Add

Show Built-in Actions

Action	Deadzone	
left	0.2	
A (Physical)		
right	0.2	
D (Physical)		
down	0.2	
S (Physical)		
up	0.2	
W (Physical)		
shoot	0.2	
Left Mouse Button - All Devices		
click	0.2	
Left Mouse Button - All Devices		
next_tool	0.2	
Mouse Wheel Up - All Devices		
previous_tool	0.2	
Mouse Wheel Down - All Devices		
sprint	0.2	
Shift (Physical)		

Close



```
1  extends CharacterBody2D
2
3
4  const SPEED = 5.0
5  var _direction: Vector2 = Vector2.ZERO
6  |
7
8  func _physics_process(delta: float) -> void:
9      » # preluam input de la tastatura -> returneaza un vector in 2D
10     » _direction = Input.get_vector("left", "right", "up", "down")
11     » # accesam proprietatea 'position' a nodului CharacterBody2D
12     » self.position = position + _direction * SPEED
13     » print(position)
14
```



Velocity

În general, nu schimbăm poziția nodului, ci proprietatea velocity. Dacă schimbăm poziția, nodul se teleportează și ar putea să ignore coliziunile cu alte obiecte.

De asemenea există metoda `move_and_slide()` care simplifică coliziunile și face calculele de fizică în backend-ul engine-ului în C++ și trebuie apelată pentru a mișca caracterul.


```
1  extends CharacterBody2D
2
3
4  const SPEED = 100.0
5  var _direction: Vector2 = Vector2.ZERO
6
7
8  func _physics_process(delta: float) -> void:
9      # preluam input de la tastatura -> returneaza un vector in 2D
10     _direction = Input.get_vector("left", "right", "up", "down")
11     # accesam proprietatea 'position' a nodului CharacterBody2D
12     velocity = _direction * SPEED
13     move_and_slide()
14
```

```
1  func _physics_process(delta: float) -> void:
2      # preluam input de la tastatura -> returneaza un vector in 2D
3      _direction = Input.get_vector("left", "right", "up", "down")
4      # accesam proprietatea 'position' a nodului CharacterBody2D
5      velocity = _direction * SPEED
6      move_and_slide()
```



Lifecycle-ul unui script

În Godot există trei metode fundamentale care rulează automat(există mai multe dar acestea sunt cele mai folosite):

_ready() → Se apelează o singură dată, automat, când nodul este instanțiat(intră în scenă).

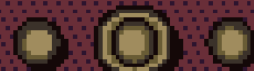
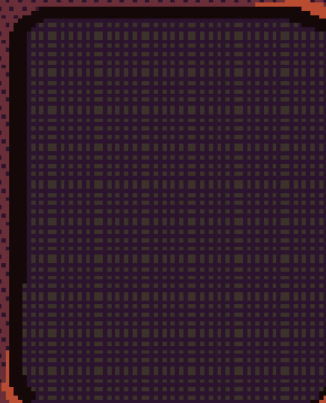
- este folosită pentru a seta starea inițială a unui nod.

_process(delta) → Se apelează la fiecare cadru(frame) desenat pe ecran.

- este folosită pentru logica vizuală.

_physics_process(delta) → Se apelează de un număr fix de ori pe secundă(ex: 60 FPS).

- este folosită pentru logica de mișcare, gravitația, coliziuni, etc.



Delta time

Se referă la diferența de timp dintre cadrul trecut și cadrul curent.

Este o variabilă folosită pentru a ne asigura că obiectele din joc se mișcă cu aceeași viteză indiferent de cât de performant este hardware-ul jucătorilor.

Warning

`move_and_slide()` automatically includes the timestep in its calculation, so you should **not** multiply the velocity vector by `delta`. This does **not** apply to `gravity` as it is an acceleration and is time dependent, and needs to be scaled by `delta`.

Revenind la scripting...

GScript este un limbaj dynamically typed → atribuirea unui tip unei variabile este opțional, dar pentru performanță, este recomandată atribuirea unui tip oricărei variabile mereu.



```
1  extends CharacterBody2D
2
3
4  var num = "123"
5  var numt: String = "123"
6  |
7  const speed: int = 100.0
8  const SPEED = 100.0
9
```

Variabile `@export` și `@onready`

Variabilele `@export` permit modificarea valorilor din editor și modificarea la runtime.

Variabilele `@onready` → în general folosite pentru referițe la alte noduri.


```

1  extends CharacterBody2D
2
3
4  @export var _speed: int = 100
5  @export var _collision_shape: CollisionShape2D
6  @onready var _sprite_2d: Sprite2D = $Sprite2D
7  var _direction: Vector2 = Vector2.ZERO
8
9
10 func _physics_process(delta: float) -> void:
11     # preluam input de la tastatura -> returneaza un vector in 2D
12     _direction = Input.get_vector("left", "right", "up", "down")
13     # accesam proprietatea 'position' a nodului CharacterBody2D
14     velocity = _direction * _speed
15     move_and_slide()
16

```

< 3 100 % 5 : 47 Tabs

[Ignore] Line 5 (UNUSED_PRIVATE_CLASS_VARIABLE): The class variable "_collision_shape" is declared but never used in the class.

[Ignore] Line 6 (UNUSED_PRIVATE_CLASS_VARIABLE): The class variable "_sprite_2d" is declared but never used in the class.

[Ignore] Line 10 (UNUSED_PARAMETER): The parameter "delta" is never used in the function

Player

Filter Properties

player.gd

Speed 100

Collision Shape Assign...

CharacterBody2D

Motion Mode Grounded

Up Direction x 0.0 y -1.0

Slide on Ceiling ☒ On

Floor

Moving Platform

Collision

CollisionObject2D

Disable Mode Remove

Collision

Input

Node2D

Transform

CanvasItem



Exercițiu: rulați jocul și modificați variabila speed și mișcați caracterul.

Să adăugăm animații...

În Godot, avem 3 noduri pentru animații:

- AnimationPlayer
- **AnimatedSpritd2D** → Cel pe care îl vom folosi noi
- AnimationTree

Asset pack download - când folosiți asset-uri externe pentru jocuri pe care plănuți să le vindeți verificați licența



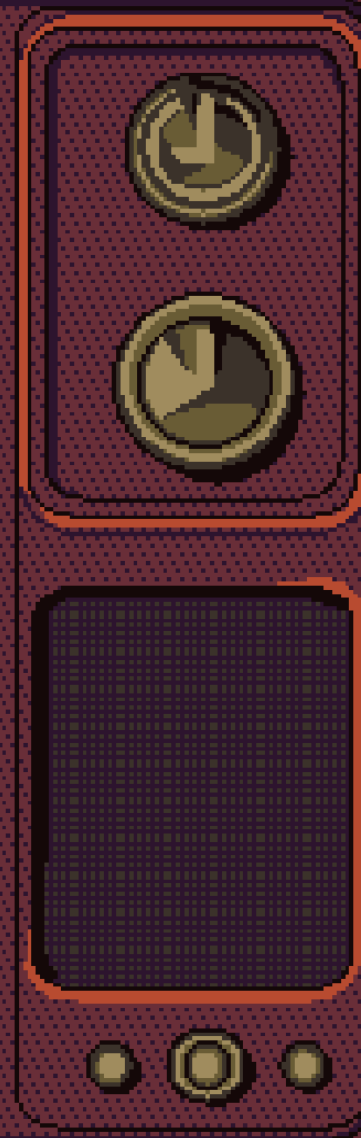
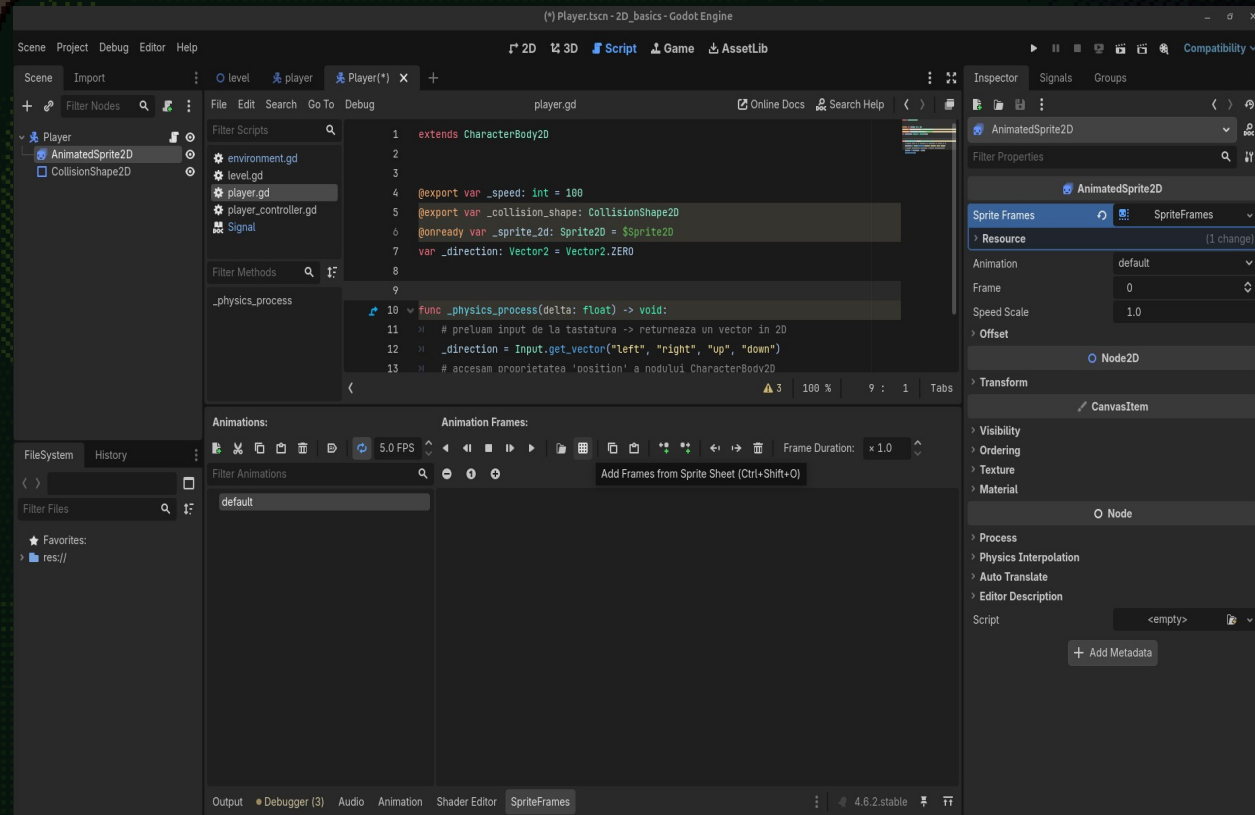


Exercițiu: schimbați nodul Sprite2D → AnimatedSprite2D

Hint: Click dreapta pe nodul Sprite2D ...

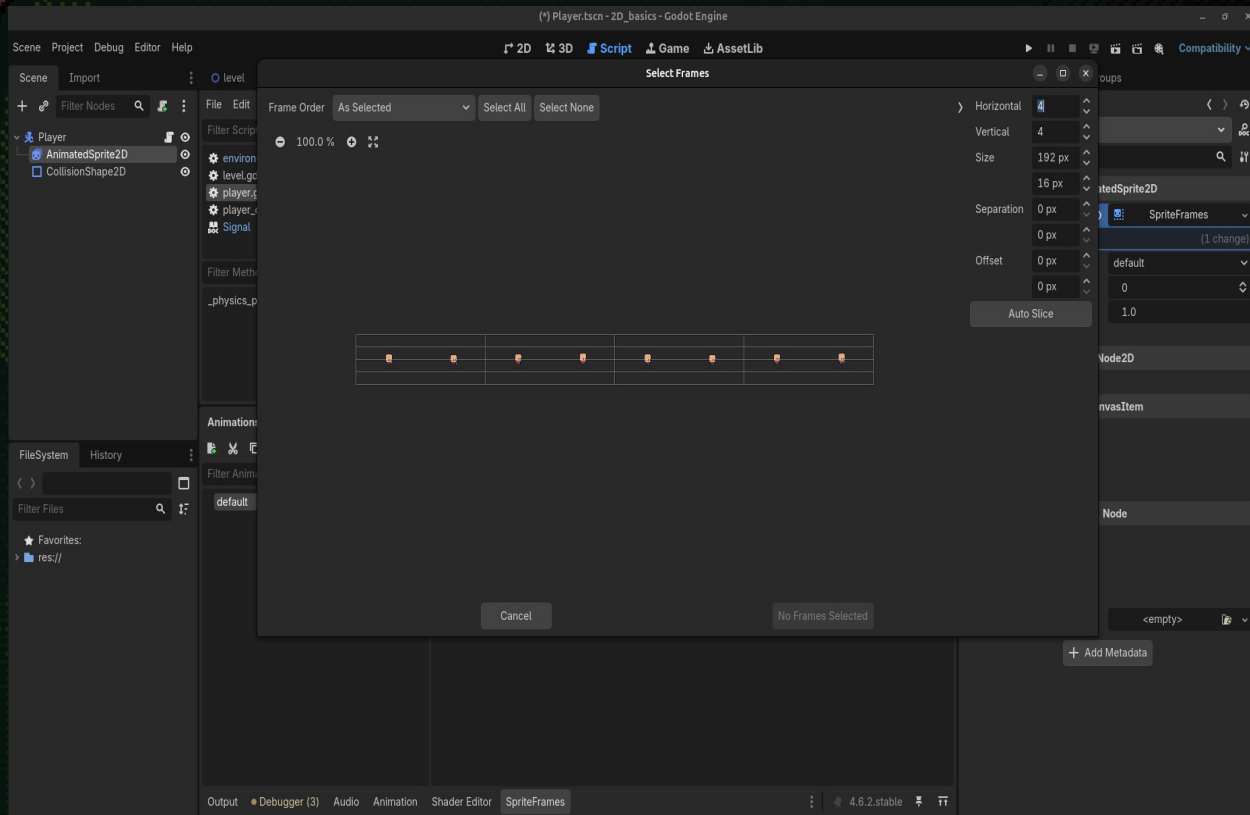


Având nodul `AnimatedSprite2D` selectat → creați o resursă nouă de tip `SpriteFrames` și selectați-o.





După acești pași, se va deschide meniul `SpriteFrames` unde putem să adăugăm animații noi și `spritesheet`-urile corespunzătoare(o animație 2D este doar o secvență de imagini).





Exercițiu: Schimbați valorile horizontal și vertical astfel încât să cuprindă toate frame-urile(imaginile) din spritesheet perfect → apoi selectați toate frame-urile și apăsați pe Add x frame(s)

459.5 %

CollisionShape2D

Animations:

Animation Frames:

5.0 FPS

Frame Duration: x 1.0

Filter Animations

default

0

1

2

3

4

5

6

7

A pixelated illustration of a vintage television set. The screen is dark and displays a faint, pixelated image of a person's head and shoulders. The television has a yellowish-brown frame and a control panel on the right side. The panel includes two circular knobs at the top, a rectangular speaker grille in the middle, and three small circular buttons at the bottom.

Exercițiu: Adăugați o animație de Idle, setați animația pe loop și pe autoplay, schimbați proprietatea FPS să fie egală cu numărul de frame-uri din animație

Să controlăm animațiile din cod...

De ce credeți că avem nevoie?

Răspuns:

- 0 referință către nodul `AnimatedSprite2D`
- Un mod de a accesa animațiile și a le schimba în funcție de starea jucătorului

```

1  extends CharacterBody2D
2
3
4  @export var _speed: int = 25
5  @export var animated_sprite: AnimatedSprite2D
6  var _direction: Vector2 = Vector2.ZERO
7
8
9  func _physics_process(delta: float) -> void:
10     # preluam input de la tastatura -> returneaza un vector in 2D
11     _direction = Input.get_vector("left", "right", "up", "down")
12
13     if _direction == Vector2.ZERO:
14         animated_sprite.play("idle")
15     else:
16         animated_sprite.play("walk")
17     # accesam proprietatea 'position' a nodului CharacterBody2D
18     velocity = _direction * _speed
19     move_and_slide()
20

```

Player

Filter Properties

player.gd

Speed 100

Animated Sprite AnimatedSprite2D

CharacterBody2D

Motion Mode Grounded

Up Direction x 0.0 y -1.0

Slide on Ceiling ☒ On

> Floor

> Moving Platform

> Collision

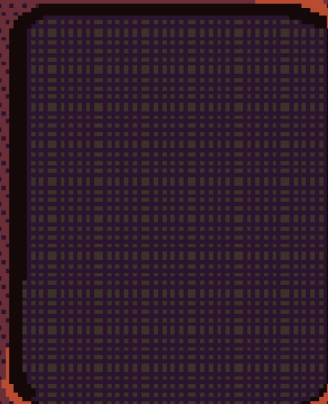
CollisionObject2D

Disable Mode Remove

> Collision

> Input

Node2D



Setări pentru jocuri cu pixel art

O să observați că texturile sunt blurate, asta se datorează setărilor de filtrare a texturilor. Putem rezolva această problemă intrând în:

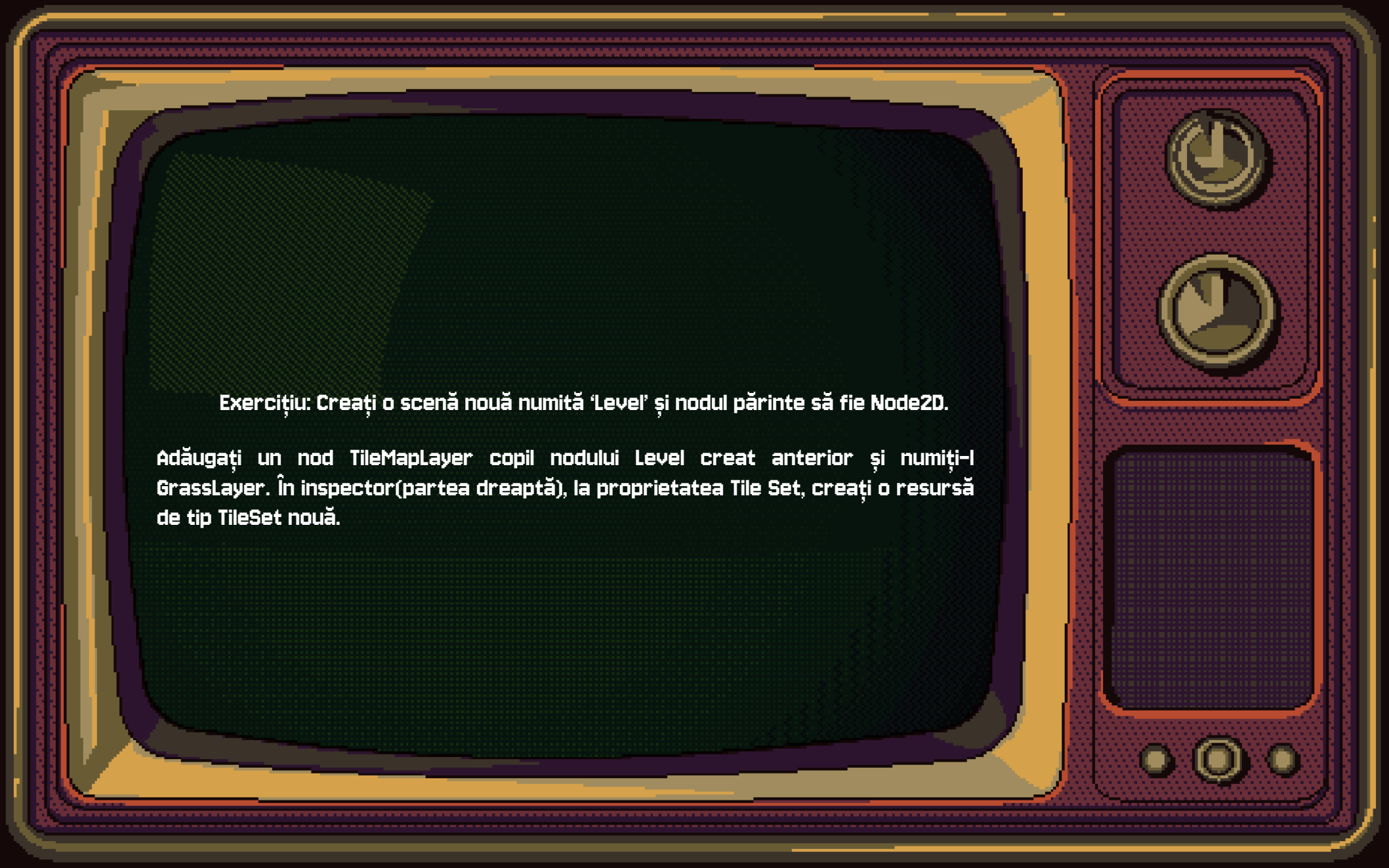
Project Settings → Advanced Settings → Rendering → Textures → Default Texture Filter → Nearest.

Mai multe informații despre texture filtering.

Tilemaps

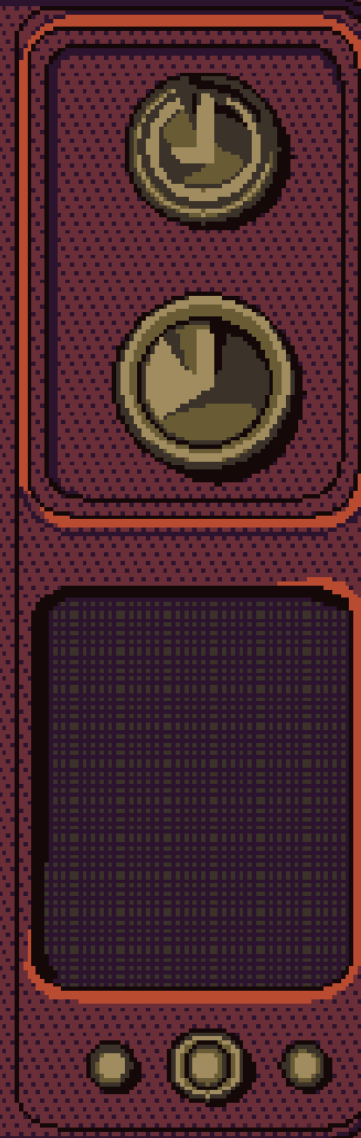
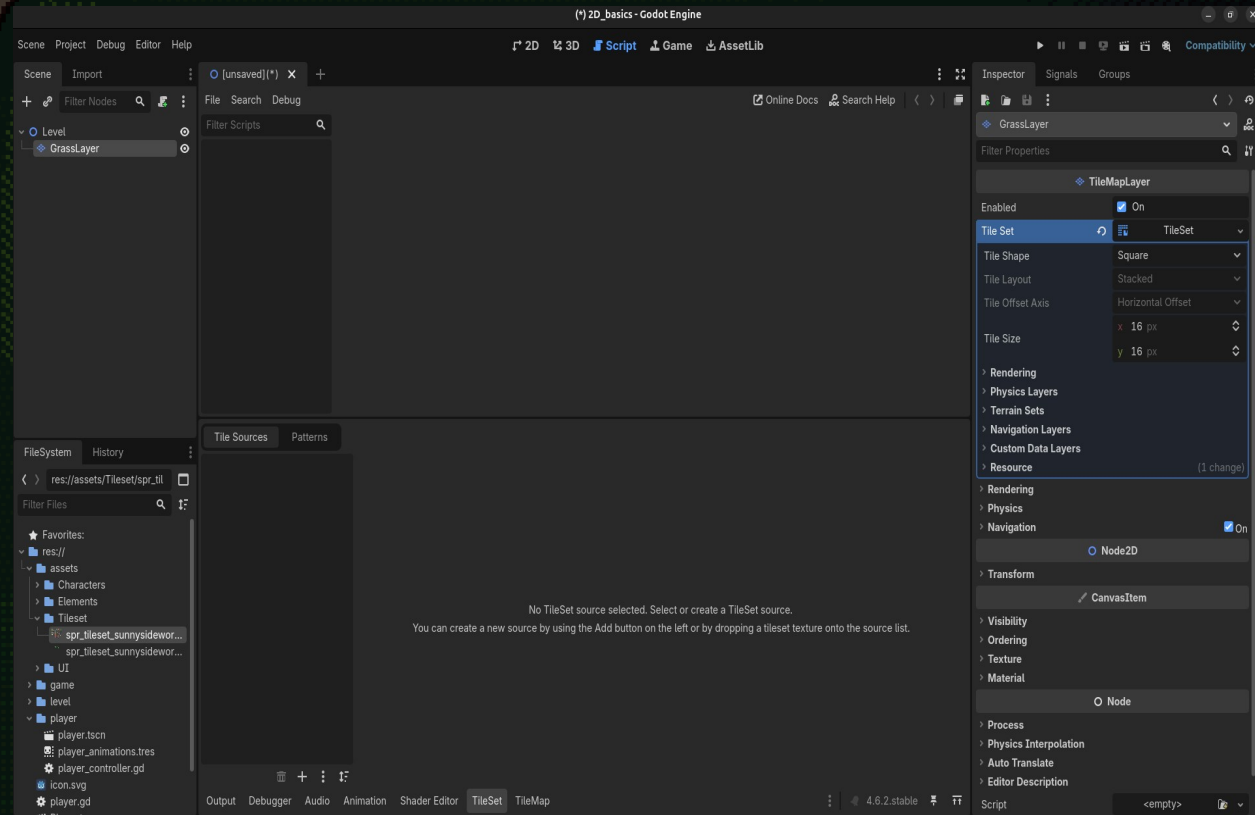
În Godot există mai multe modalități de a crea nivele. O modalitate ar fi de a crea toată arta din sprite-uri separate și să construim nivelul așa, dar noi vom folosi tilemaps.

Tilemap-urile permit organizarea nivelelor sub formă de matrice(grid-uri) și putem picta nivelul cu anumite tile-uri(texturi) dintr-un atlas(o textură care conține toate tile-urile).



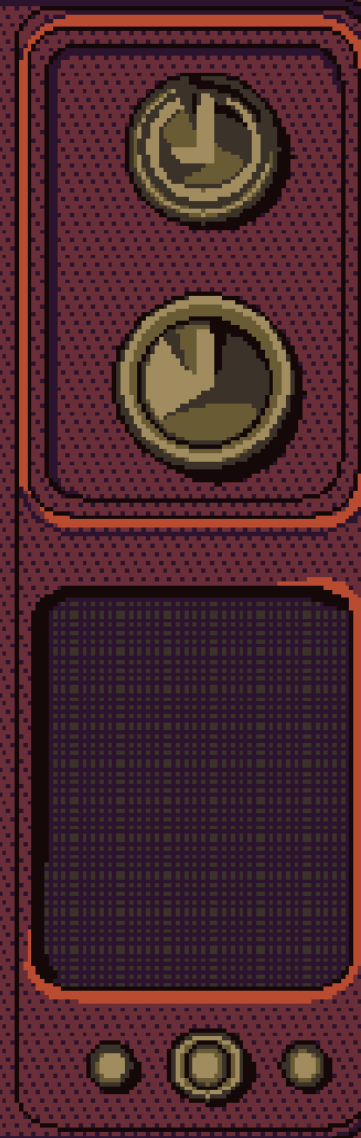
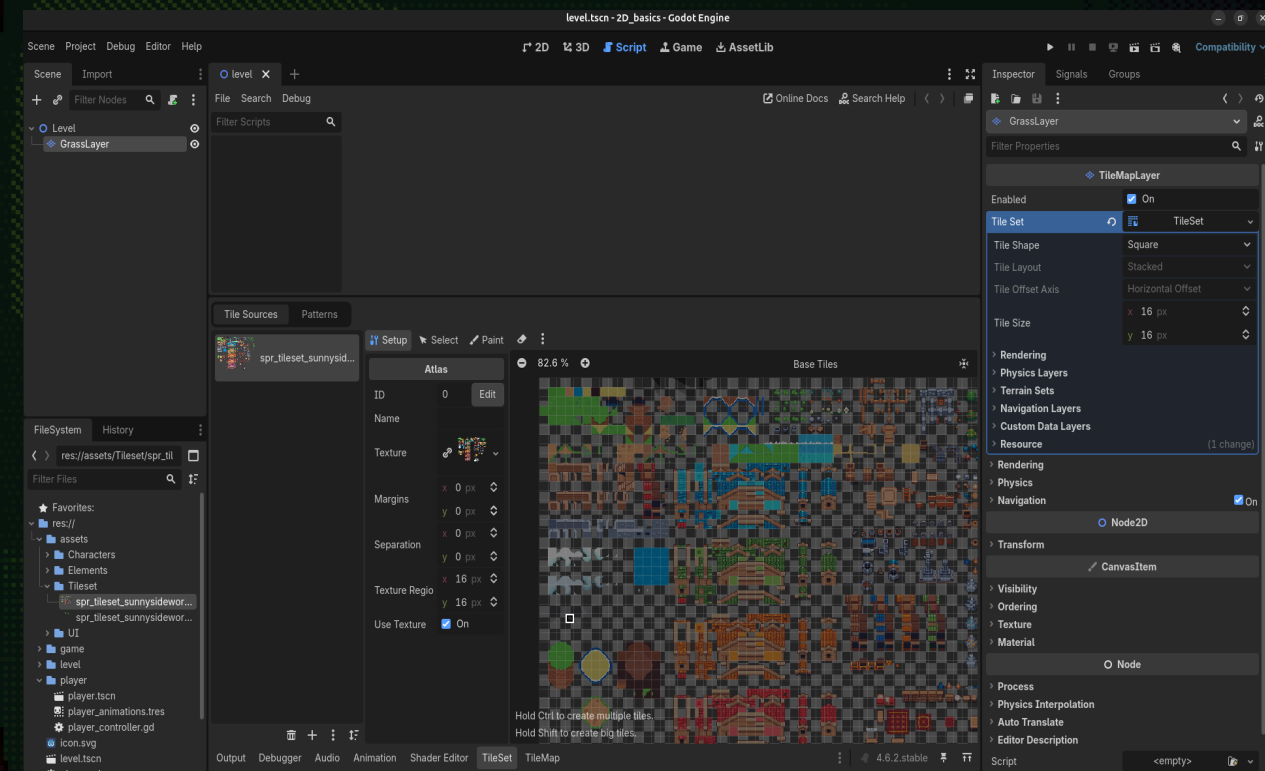
Exercițiu: Creați o scenă nouă numită 'Level' și nodul părinte să fie Node2D.

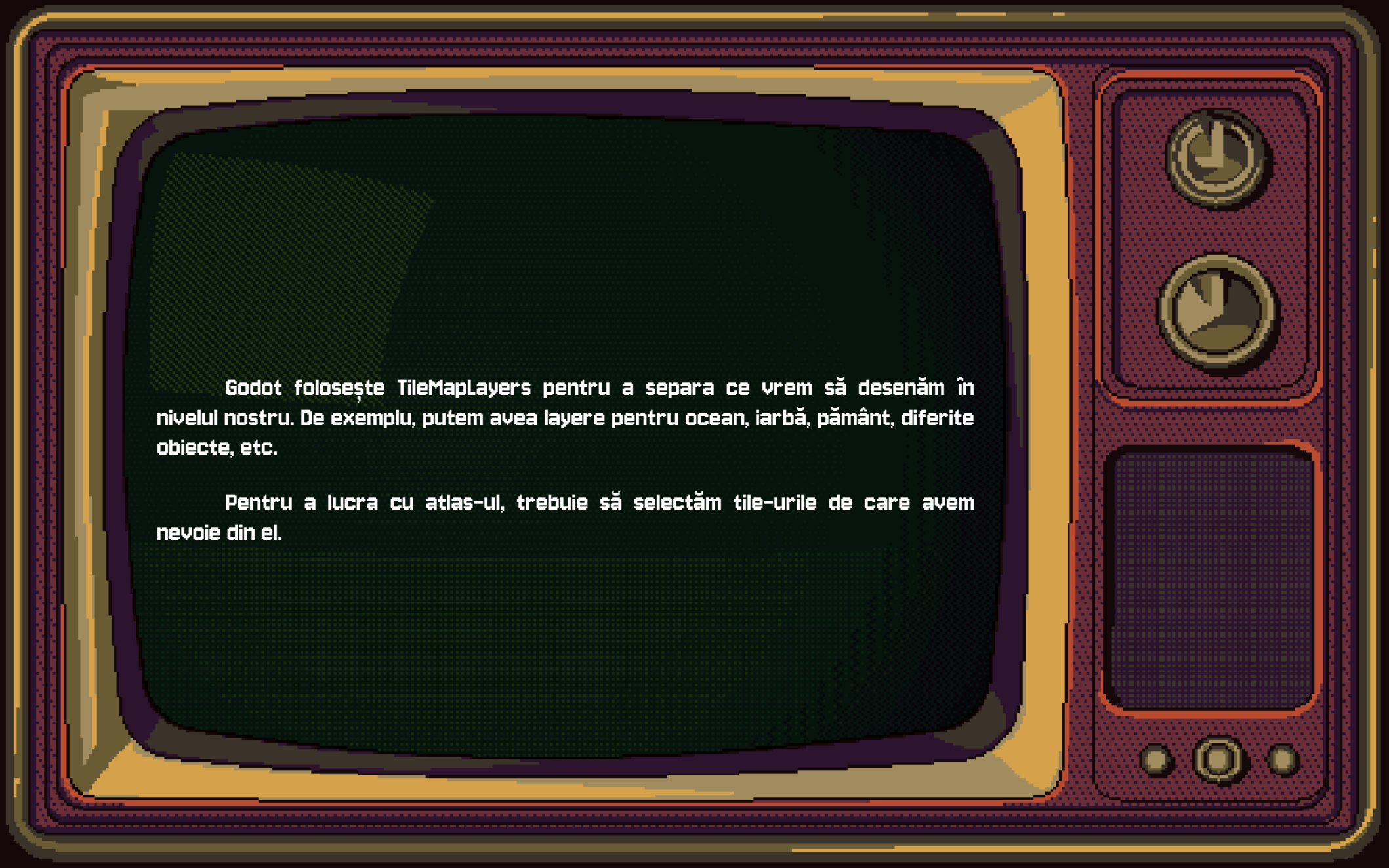
Adăugați un nod TileMapLayer copil nodului Level creat anterior și numiți-l GrassLayer. În inspector (partea dreaptă), la proprietatea Tile Set, creați o resursă de tip TileSet nouă.





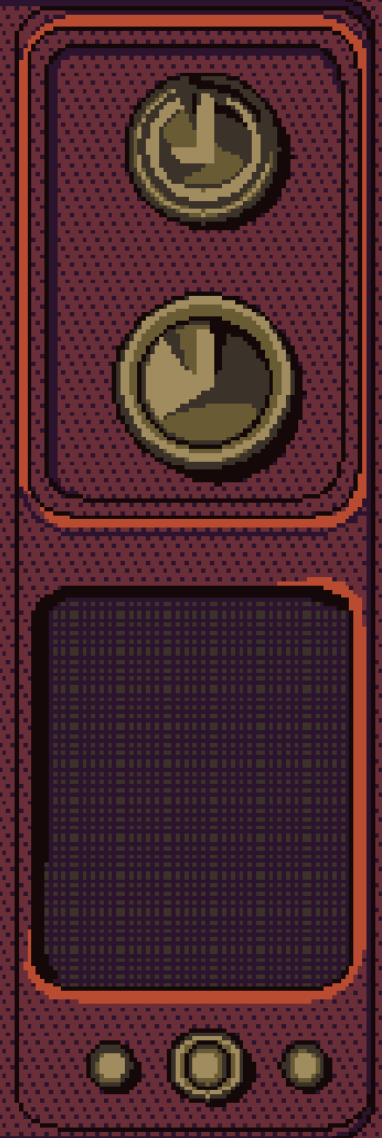
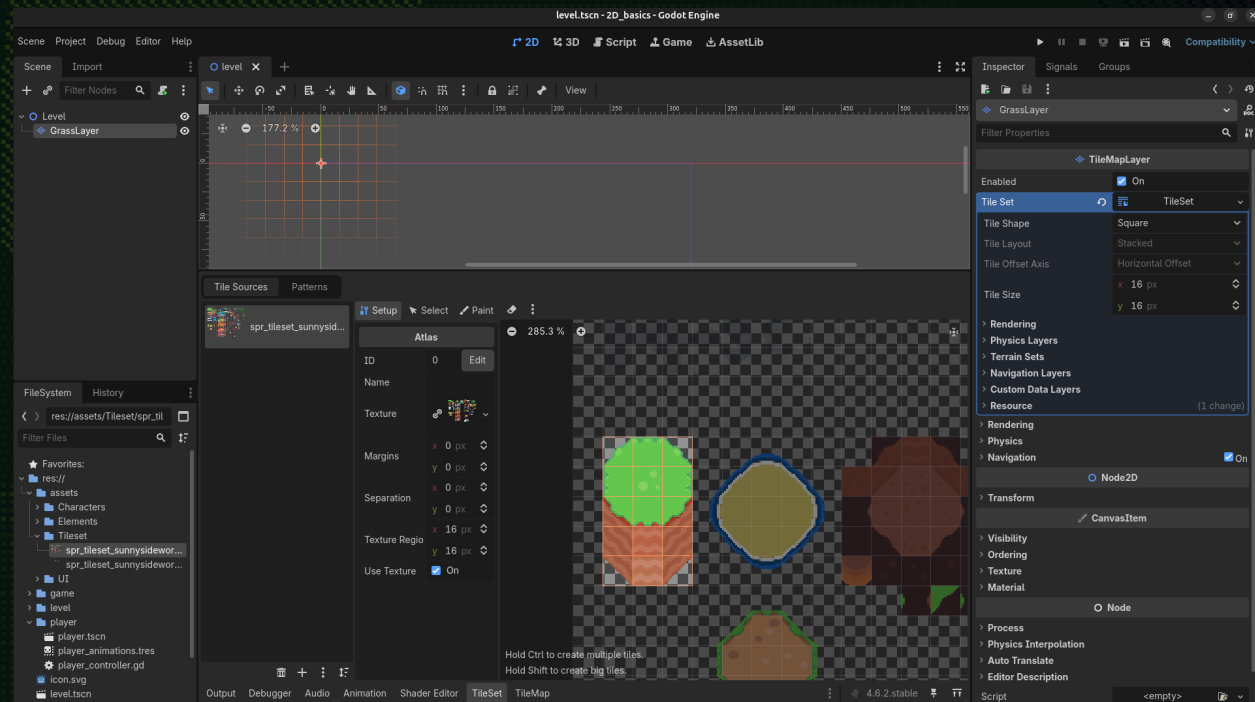
Exercițiu: În window-ul TileSet, adăugați un nou tile source Atlas și adăugați ca sursă din folder-ul assets tileset-ul downloadat în capitolul anterior.





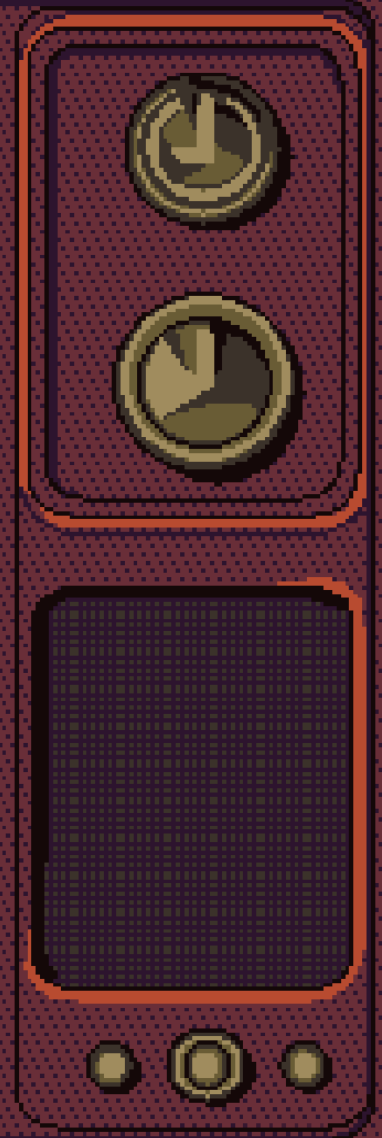
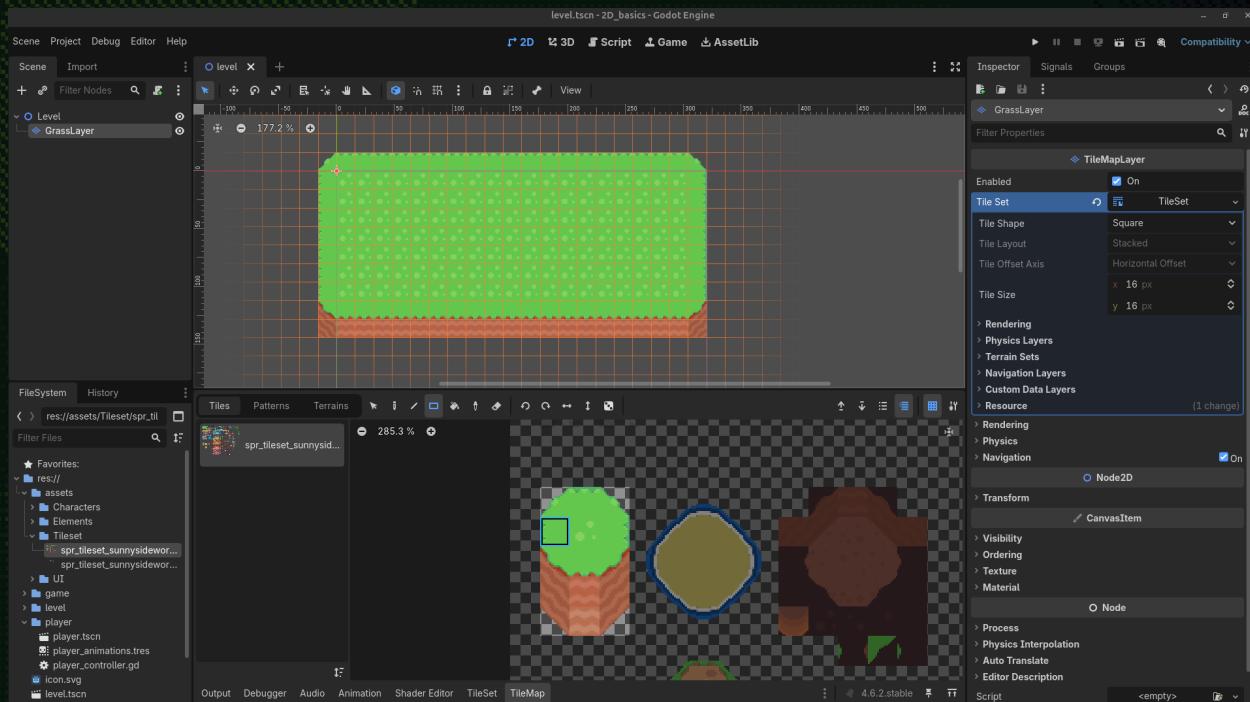
Godot folosește `TileMapLayers` pentru a separa ce vrem să desenăm în nivelul nostru. De exemplu, putem avea layere pentru ocean, iarbă, pământ, diferite obiecte, etc.

Pentru a lucra cu atlas-ul, trebuie să selectăm tile-urile de care avem nevoie din el.



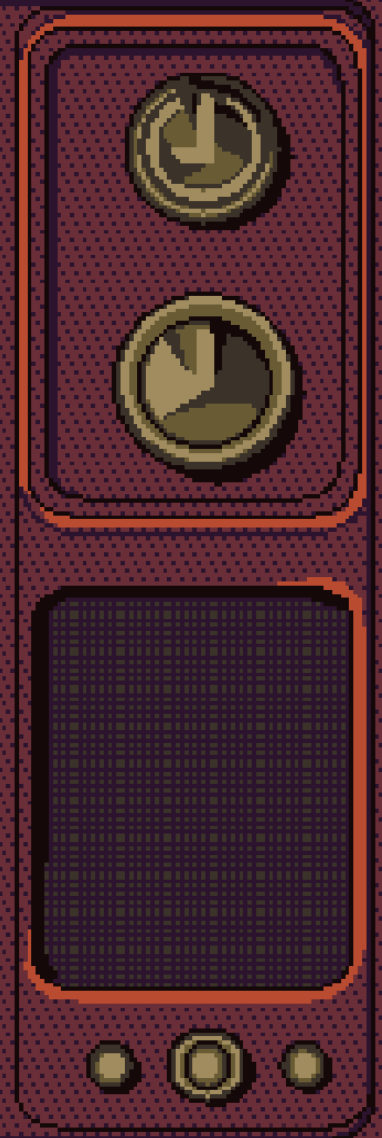
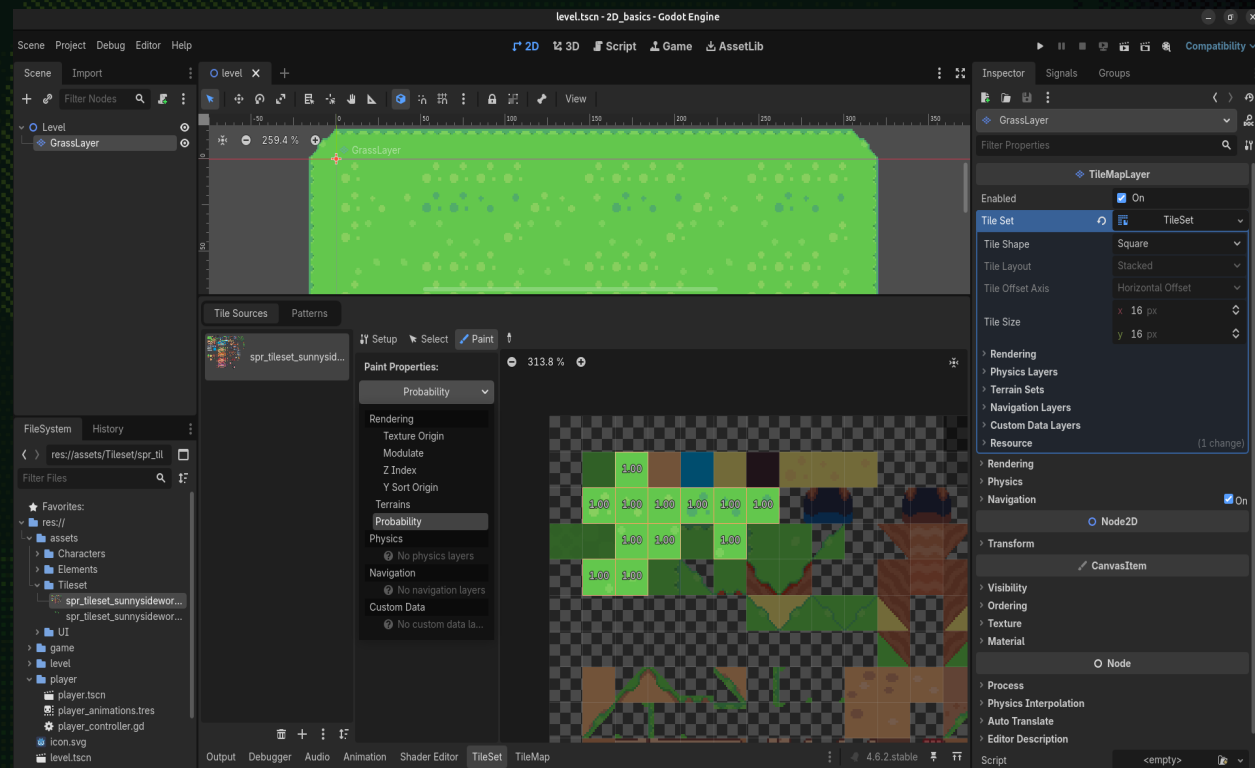


Din window-ul TileMap putem desena tile-uri din atlas pe hartă cu click stânga adăugăm tile-uri și cu click dreapta putem să le ștergem.



Tilemap paint properties

În Godot putem adăuga proprietăți tile-urilor, acestea pot fi: probabilități pentru anumite tile-uri să fie folosite când pictăm cu ele, layere de coliziune, layere de navigație pentru AI și date custom(de exemplu dacă vrem să se miște mai repede un Player pe anumite tile-uri sau să primească damage, etc).

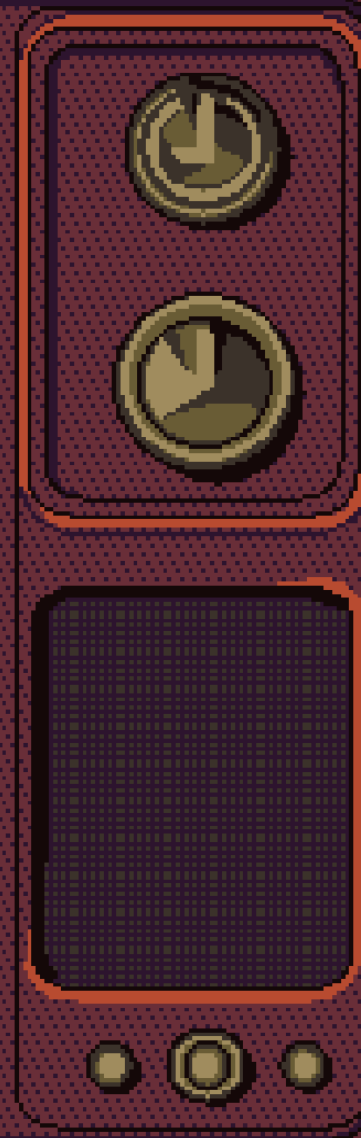
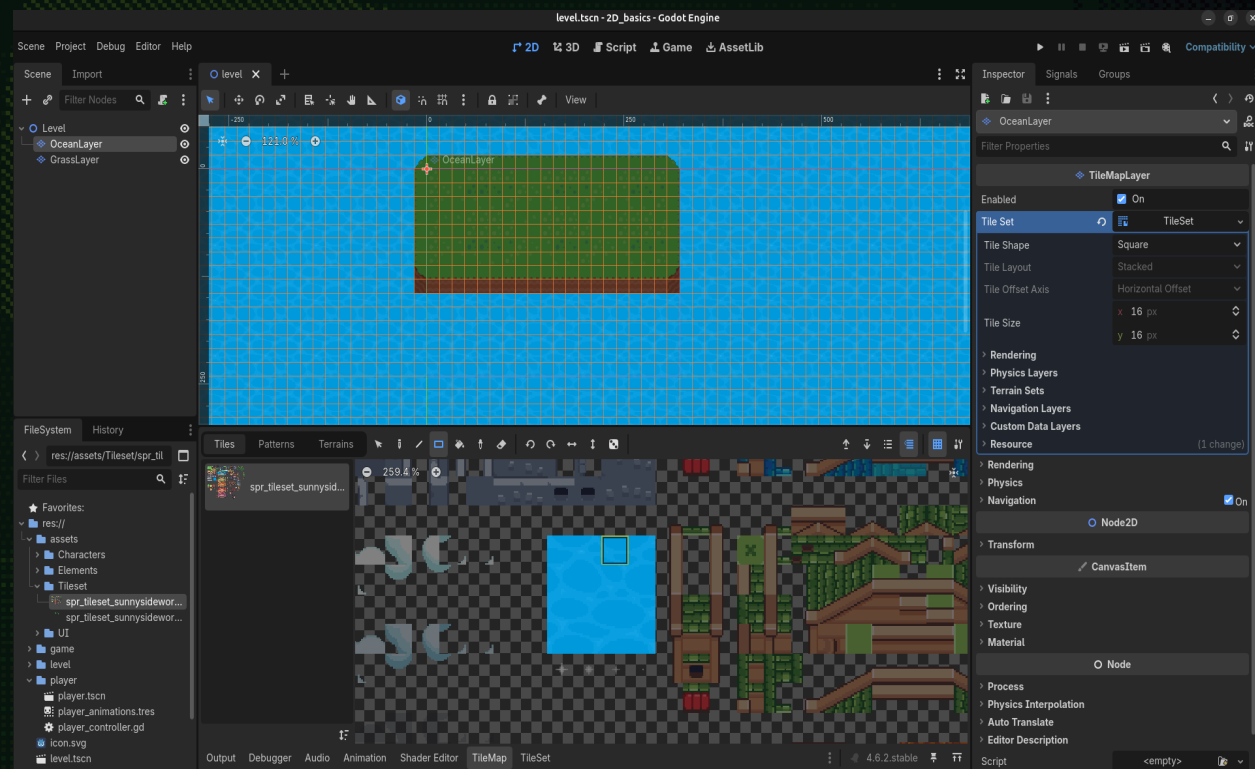




Când am pictat am folosit proprietatea `scattering(zarul)` pentru a avea tile-uri random dintre cele selectate. Am selectat mai multe tile-uri folosind `SHIFT + LEFT_CLICK`.

Exercițiu: Adăugați un `TileMapLayer` nou și numiți-l `OceanLayer` și desenați un ocean sub `GrassLayer`.

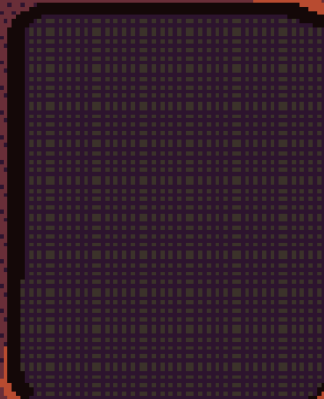
Hint: Dacă oceanul apare peste grass layer, gândiți-vă cum puteți muta nodul din window-ul Scene.

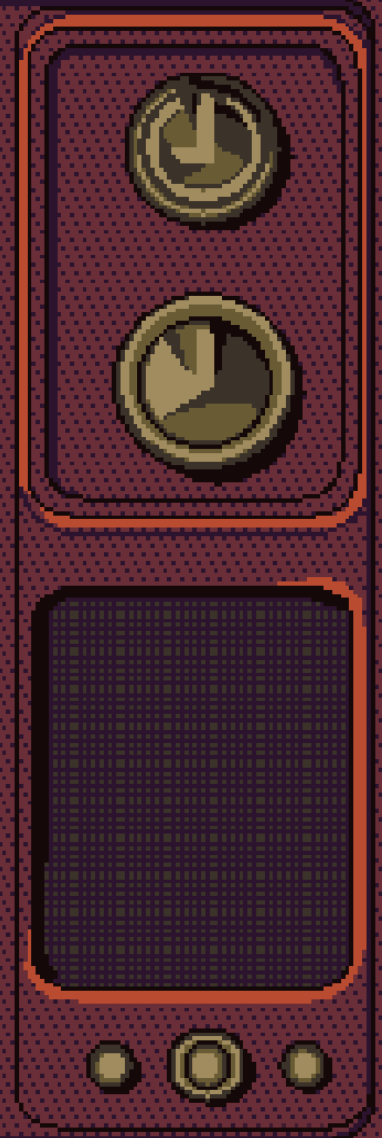
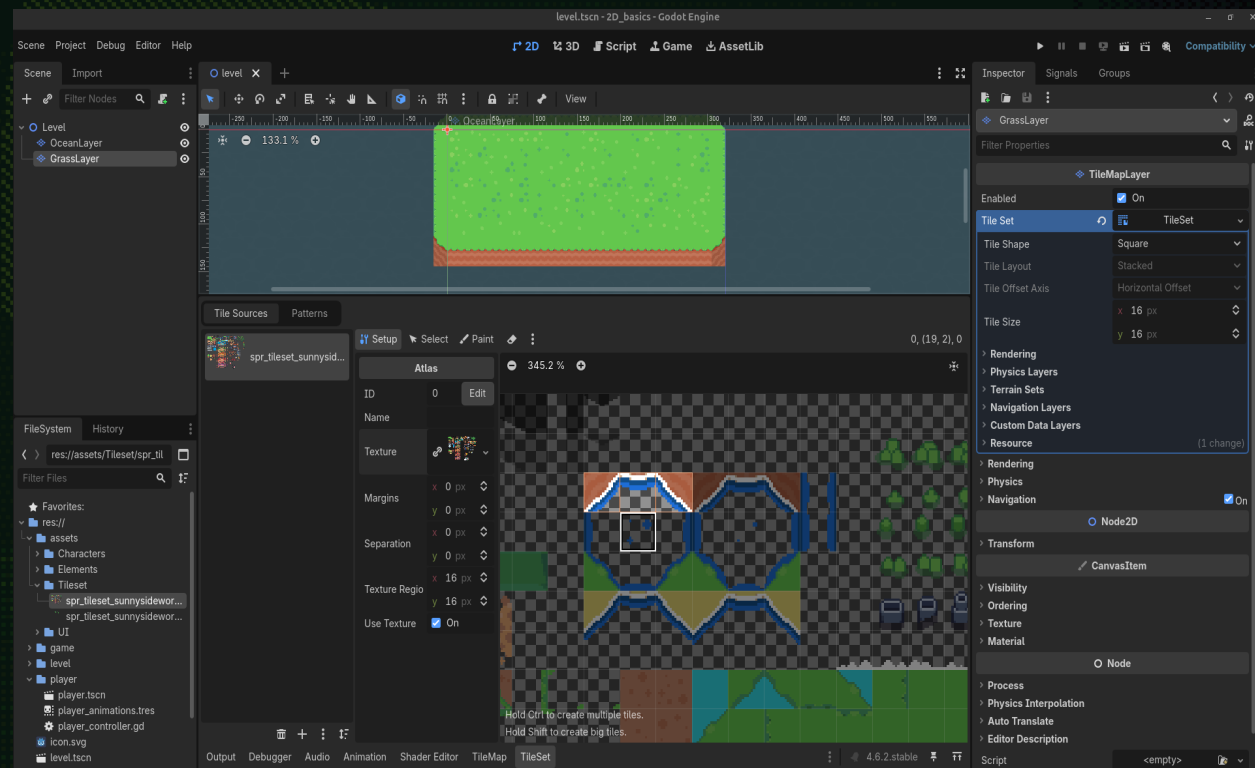


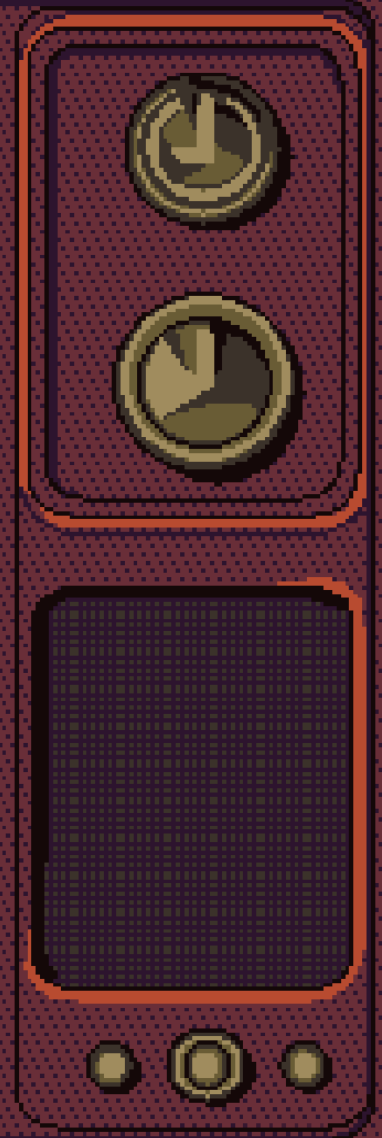
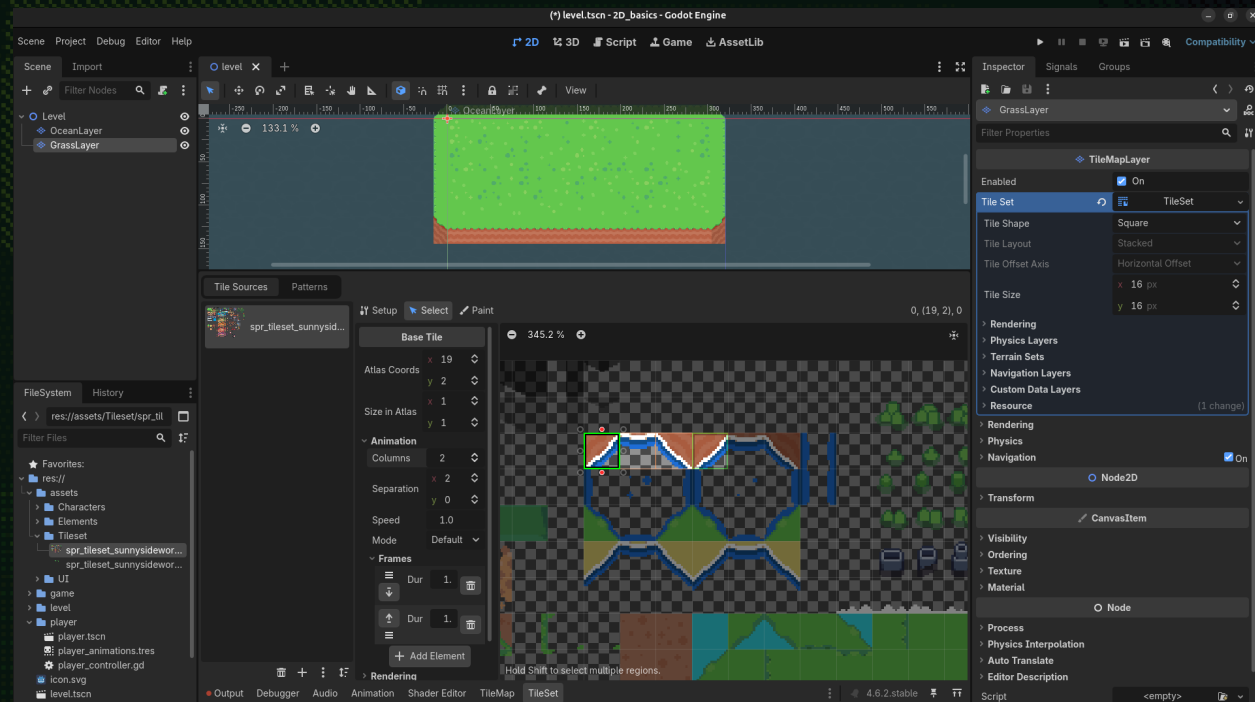
Tilemap animations

Tilemap-urile pot avea și tile-uri animate.

Reveniți la grass layer și selectați următoarele tile-uri din atlas:





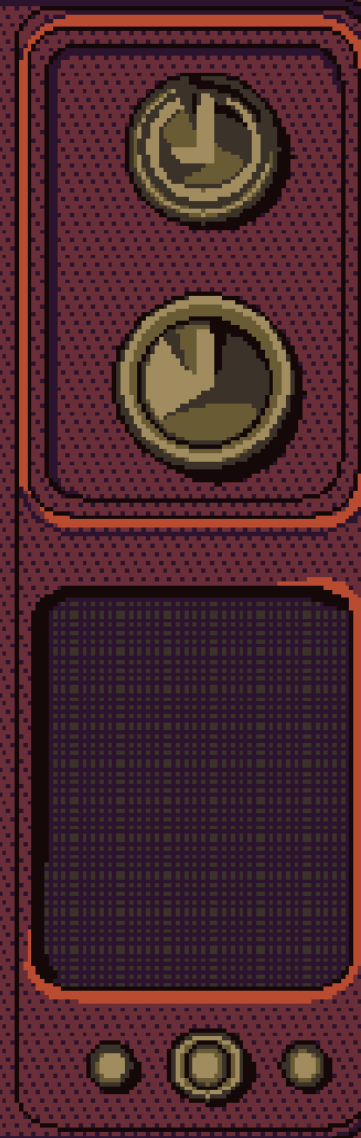
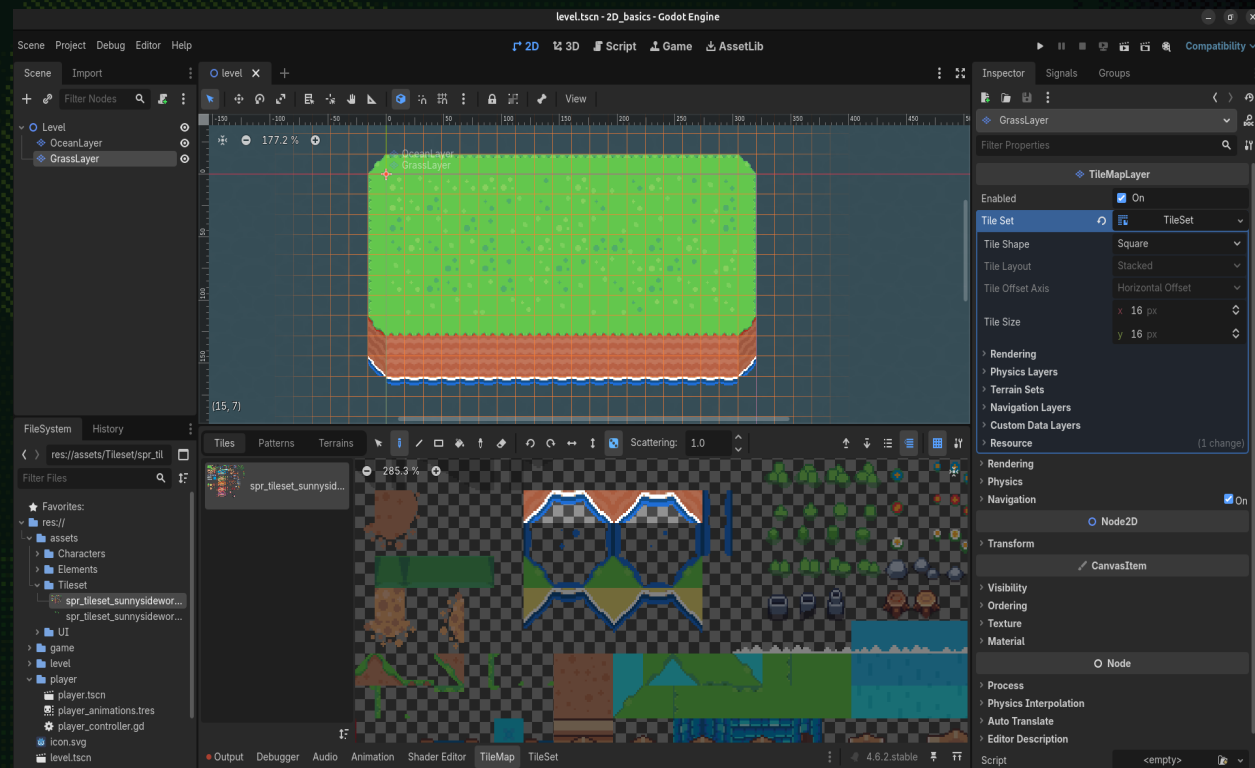




Am folosit opțiunea select pe primul tile și am select dropdown-ul animation.
În cazul acesta avem doar 2 frame-uri pentru aceste tile-uri.

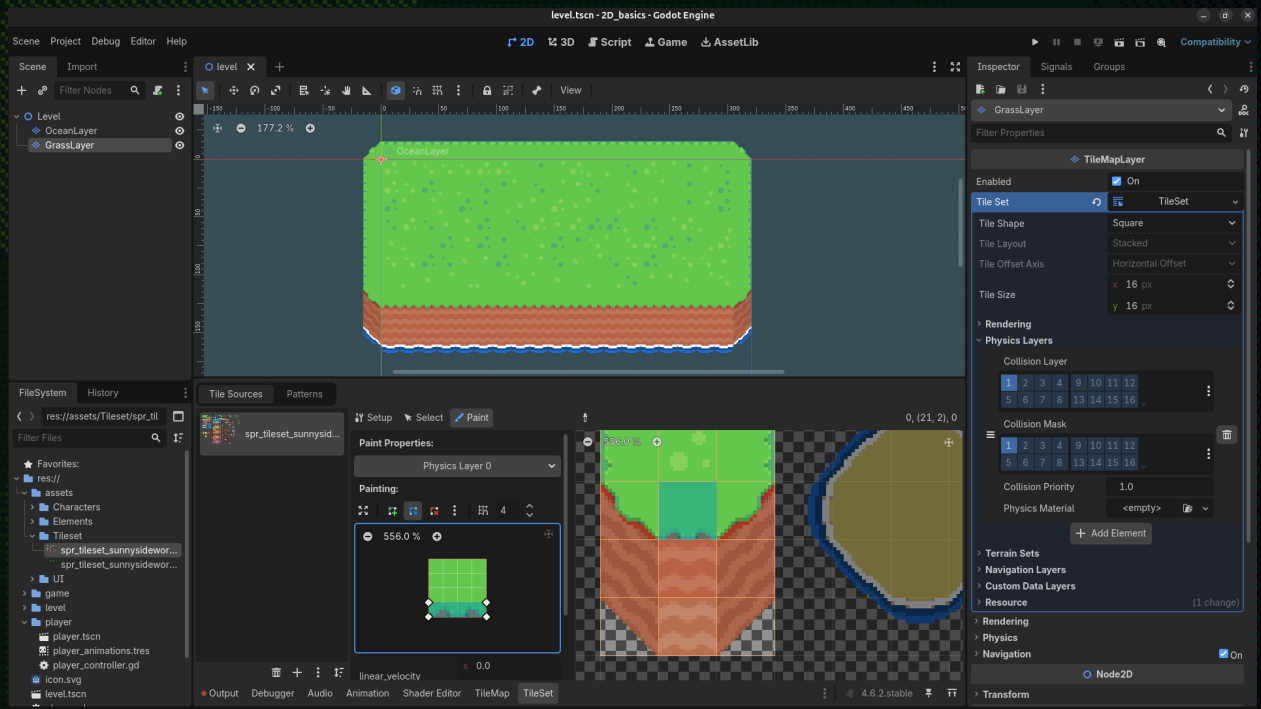


Exercițiu: Adăugați tile-urile animate sub tile-uri maro.



Adăugarea collision-urilor

Selectați GrassLayer și proprietatea tile set din inspector, apoi creați un physics layer nou. Apoi din window-ul TileSet folosim opțiunea paint cu proprietatea physics layer creată și desenăm collision shapes pentru tile-urile pe care le vrem să aibă collision detection cu Player-ul.

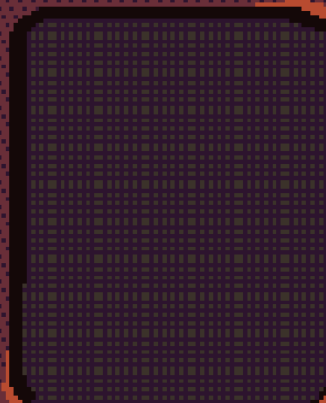


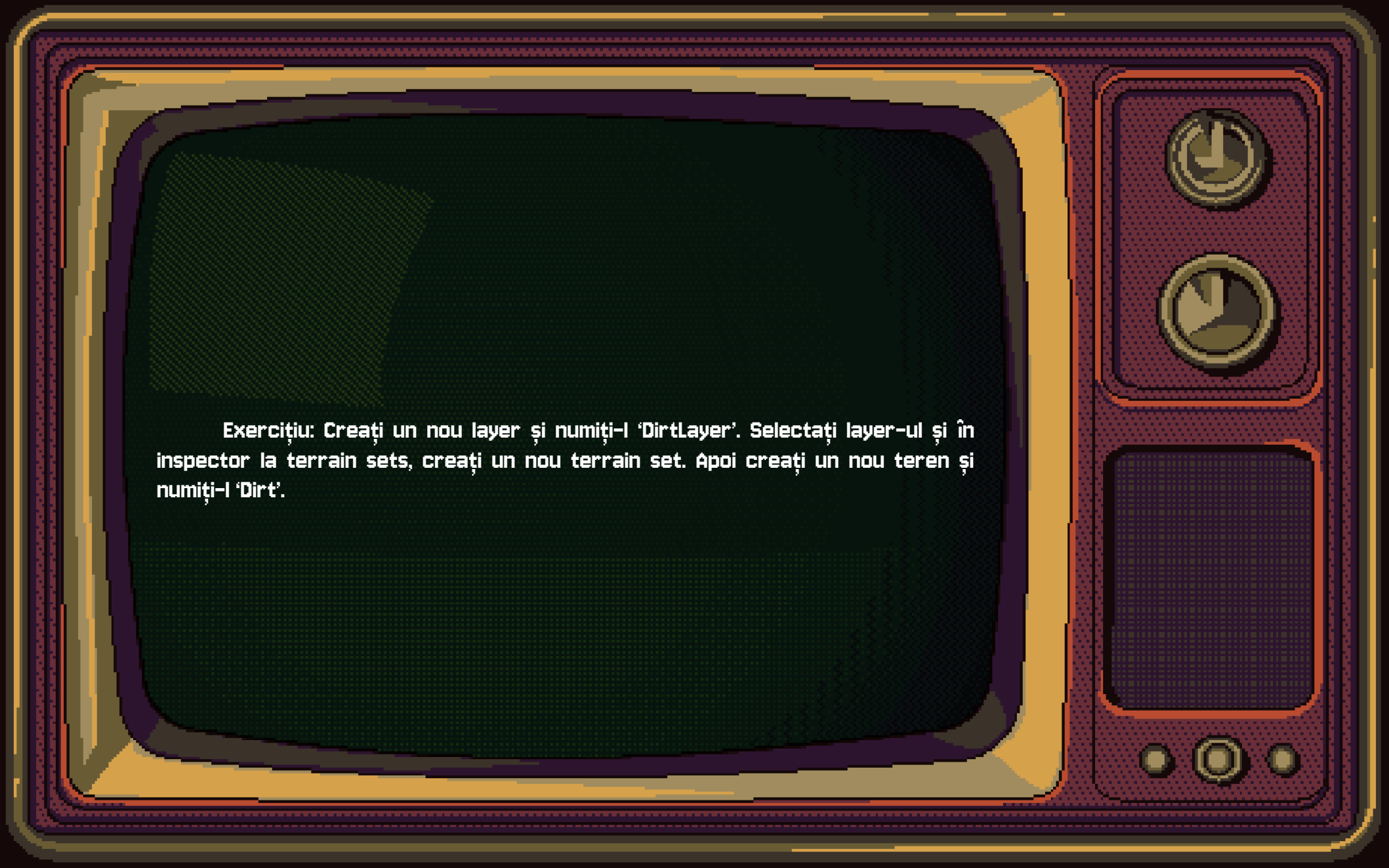
Autotiling(terrains)

Terrains sunt un sistem în Godot care permite tile-urilor de diferite feluri să se conecteze automat între ele bazat pe niște reguli. Acest mecanism permite desenarea nivelelor mult mai rapid ca atunci când amplasăm tile-uri normal.

Terrain tile-urile au un bit în centru și peering bits(biții de pe margine), fiecare bit poate să aibă un terrain. Așadar, terrains funcționează pe baza a bitmasks, similar cu sistem-ul de collisions.

Terrain set-urile au mai multe moduri de conexiune, dar noi vom folosi match cornes and sides, cel mai des folosit mod.





Exercițiu: Creați un nou layer și numiți-l 'DirtLayer'. Selectați layer-ul și în inspector la terrain sets, creați un nou terrain set. Apoi creați un nou teren și numiți-l 'Dirt'.



Pentru a folosi acest terrain, în tab-ul tileset mergem la proprietatea paint din nou și selectăm opțiunea Terrains → TerrainSet0 → Terrain → Dirt.

Tile Sources

Patterns



spr_tileset_sunnysid...

Setup

Select

Paint



0, (7, 33), 0

Paint Properties:

505.4 %

Terrains

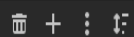
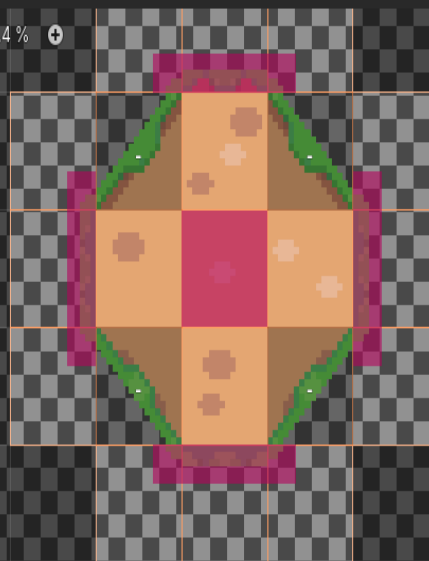
Painting:

Terrain Set

Terrain Set 0

Terrain

Dirt



Output

Debugger

Audio

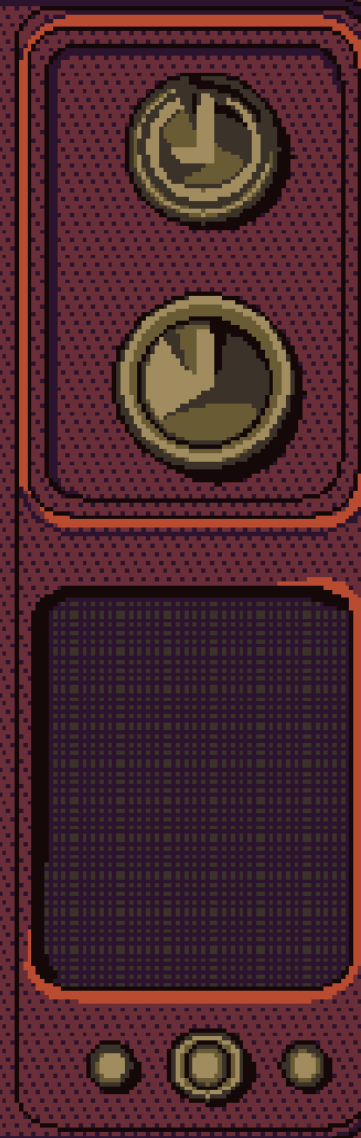
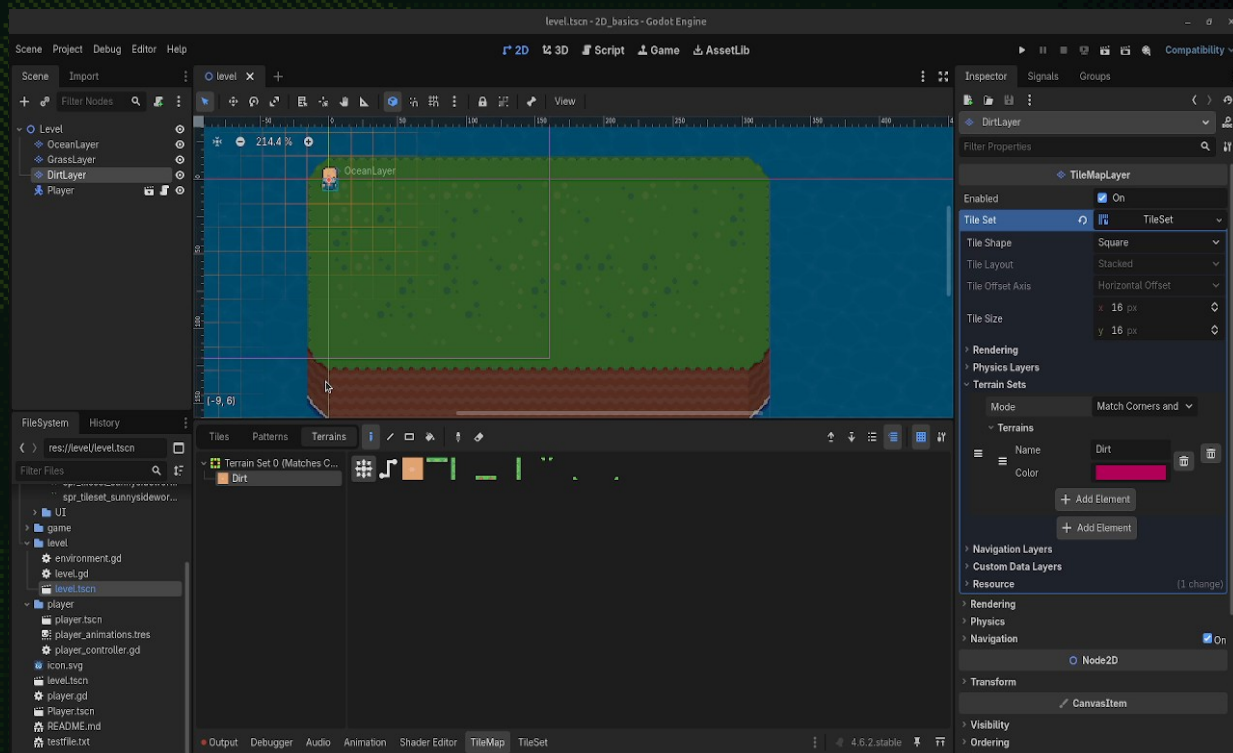
Animation

Shader Editor

TileMap

TileSet

4.6.2.stable



Temă(COMPLET OPȚIONALĂ):

- Adăugați mai multe animații
- Încercați să dați flip la sprite în funcție de direcția în care merge

hint: folosiți variabila `_direction` și `x` sau `y` pentru componenta orizontală/verticală a vectorului

- Adăugați mai multe inputuri/ stări pentru animațiile adăugate

hint: folosiți `Input.is_action_pressed("<nume_acțiune>")`

- Creați o metodă(funcție) pentru logica de animații
- Adăugați un layer nou cu flori și alte obiecte decorative

hint: folosiți-vă de randomizarea pe care v-am arătat-o în capitolele anterioare

